INTEGRATION, the VLSI journal 81 (2021) 175–194

Contents lists available at ScienceDirect

Integration, the VLSI Journal

journal homepage: www.elsevier.com/locate/vlsi

FPGA-based Physical Unclonable Functions: A comprehensive overview of theory and architectures

N. Nalla Anandakumar [a,*], Mohammad S. Hashmi [b], Mark Tehranipoor [a]

[a] Electrical & Computer Engineering Department, University of Florida, Gainesville, FL, 32611, USA
[b] School of Engineering and Digital Sciences, Nazarbayev University, Astana, 010000, Kazakhstan

ARTICLE INFO

Keywords:
Physical Unclonable Functions (PUFs)
Field-Programmable Gate Array
Hardware security
PUF applications
FPGA based PUFs

ABSTRACT

Physically Unclonable Functions (PUFs) are a promising technology and have been proposed as central building blocks in many cryptographic protocols and security architectures. Among other uses, PUFs enable chip identifier/authentication, secret key generation/storage, seed for a random number generator and Intellectual Property (IP) protection. Field Programmable Gate Arrays (FPGAs) are re-configurable hardware systems which have emerged as an interesting trade-off between the versatility of standard microprocessors and the efficiency of Application Specific Integrated Circuits (ASICs). In FPGA devices, PUFs may be instantiated directly from FPGA fabric components in order to exploit the propagation delay differences of signals caused by manufacturing process variations. PUF technology can protect the individual FPGA IP cores with less overhead. In this article, we first provide an extensive survey on the current state-of-the-art of FPGA based PUFs. Then, we provide a detailed performance evaluation result for several FPGA based PUF designs and their comparisons. Subsequently, we briefly report on some of the known attacks on FPGA based PUFs and the corresponding countermeasures. Finally, we conclude with a brief overview of the FPGA based PUF application scenarios and future research directions.

1. Introduction

The annual revenue of FPGA market was valued at USD 6.4 billions in 2018 and is expected to reach USD 11.0 billions by 2025 as per a new research report by Markets and Markets Inc [1]. One important reason for the rapid growth of FPGA market can be attributed to the increasing fabrication cost of ASICs by the advanced manufacturing process technology. For example, the chip design cost in 28 nm process technology is estimated to be around USD 170 millions which is almost double as compared to the cost in 45 nm technology [2]. On the other-hand, the cost associated with entry to the FPGA market is low and this facilitates design and validation of products with advanced process technology without investing heavily into the physical design and the production cost. Furthermore, compared to ASICs, the FPGAs offer additional advantages in terms of short time-to-market, lower design costs and availability of third party IPs. Due to these advantages, FPGAs are considered the platform of choice for many applications in wide variety of domains including medical devices, remote sensing, military and aerospace, government systems, automotive and consumer electronics, industrial control systems, etc. [3–6]. However, the multitude of associated benefits cannot conceal the constraints associated with the FPGAs in these applications.

For example, cloning is a serious problem with the FPGAs because the act of copying or tampering with the FPGA based designs are relatively easy. Since many types of FPGAs are volatile, their configuration has to be reloaded at every powerup phase. Therefore, any FPGA design contained in *bitfile* is often stored on an external memory such as EEPROM or flash on the same PCB as the FPGA. During loading of the *bitfile*, an adversary who has physical access to the device can eavesdrop on the bus. Then, this *bitfile* can easily be copied and illegally configured onto other FPGAs, which is equivalent to cloning the original (IP) design. To address these problems, FPGA manufacturers are providing the possibility to use encrypted *bitfile* with the concept of authentication [7]. In that case, a unique private/authenticate key has to be stored on-chip to be able to decrypt the *bitfile*. However, most of the FPGAs do not contain non-volatile memory to store this private key. In general, non-volatile storage like EEPROM has to be added [7]. But this solution is increase the size and cost of the product. Moreover, for sensitive designs, a permanently stored secret key is vulnerable to probing attacks and possibly other side-channel attacks. The PUFs address this shortcomings of the digital key storage (i.e., since the key is not permanently stored in non-volatile storage, but only appears in volatile memory when required for operation) by relying on the

* Corresponding author.
E-mail addresses: nallananth@gmail.com (N.N. Anandakumar), mohammad.hashmi@nu.edu.kz (M.S. Hashmi), tehranipoor@ece.ufl.edu (M. Tehranipoor).

https://doi.org/10.1016/j.vlsi.2021.06.001
Received 26 November 2020; Received in revised form 10 June 2021; Accepted 24 June 2021
Available online 17 July 2021

secrets generated by the inherent and unclonable unique mesoscopic characteristics of the physical phenomena [8,9], which is extremely hard or impossible to clone. The physical properties of each silicon device are determined by using a set of challenges (inputs) and set of responses (outputs). PUFs can be used different type of security protocols, such as unique identifiers, secret keys, device authentications, IP protection and pseudo random bit generators (PRNG). A number of PUFs have been realized in ASIC and specifically in FPGA. The pervasive use of FPGAs in embedded applications and quick product customization suggests that it is desirable for PUFs to be realized on FPGAs. Some interesting applications of FPGA-based PUFs could be found in secret key generation [10–12], random number generation [13,14], FPGA IP protection [15,16], device identification [17], chip authentication [11,13,18], key exchange/agreement protocols [19–21], anti-counterfeiting [22], and IoT security [23–26], etc. Current FPGA market leaders, namely Xilinx [27], Intel (former Altera) [28] and Microsemi [29], have already started to integrate PUFs into their mass products for security purposes. Currently, the Xilinx UltraScale+ FPGAs enables the user to implement soft PUF IP cores (used as PUF key) to protect the user key (red key) during configuration [27]. Similarly, Intel Stratix 10 FPGAs enable user access to a PUF as part of the FPGA device configuration process for user key protection and device identification purposes [27]. Furthermore, selected Microsemi PolarFire FPGAs [29] can be utilized as a Root of Trust for secure authentication and also used to protect the customer's IP. We can be also purchased Soft PUFs from many third-party developers, such as Intrinsic-ID Inc [30], Lewis Innovative Technology [31], QuantumTrace [32] and Helion Technology Ltd [33], etc.

In this article, we report a literature survey of the state-of-the-art of FPGA based PUFs, highlight some recent research advances associated with FPGA based PUFs, and discuss their some key applications. The remainder of this paper is organized as follows: The detailed PUF taxonomy is presented in Section 2, whereas Section 3 describes FPGA-based PUF designs, and the performance evaluation results of recently reported FPGA based PUF realizations and their comparisons are given in Section 5. Some of the well-known attacks to FPGA-based PUF and the corresponding countermeasures are briefly discussed in Section 4, whereas the FPGA based PUF application scenarios are included in Section 6. A number of future research directions are discussed in 7 and finally conclusions are drawn in Section 8.

2. A PUF primer

A PUF is a function that is realized by a physical system and is easy to evaluate but the physical system is hard or impossible to clone [8]. They have the ability to extract unique signature, from the intrinsic process variability of silicon devices, using a challenge and response mechanism. The basic functionality of a PUF f can be described mathematically as a mapping $R = f(C)$, where C is a challenge (an external stimulus) applied and R is the response (output) generated by the PUF (Fig. 1a). The primary security-related features of PUFs are outlined in (Fig. 1b–g) [34]. These are succinctly explained below:

Reproducible: A response $R = f(C)$ can be reproduced up to a correctable error (Fig. 1b).

Unique: The function $f(C)$ contains some information about the identity of the physical entity embedding f (Fig. 1c).

Unclonable: When given f, it remains hard to construct a procedure g (i.e., physical entity) such that $f(C) \approx g(C)$ up to a small error (Fig. 1d).

One-way: When given only R and f, it is hard to find C such that $f(C) = R$ (Fig. 1e).

Unpredictable: Given only a set of CRPs Q, where $R = f(C)$ and $(C, R) \in Q$, it remains hard to compute $R' = f(C')$, where C' is a random challenge and $(C', R') \notin Q$ (Fig. 1f).

Tamper-evident: When the physical entity that constitutes f is altered, this transforms f into f', such that $f(C') \neq f(C)$ with high probability and also not up to a correctable error (Fig. 1g).

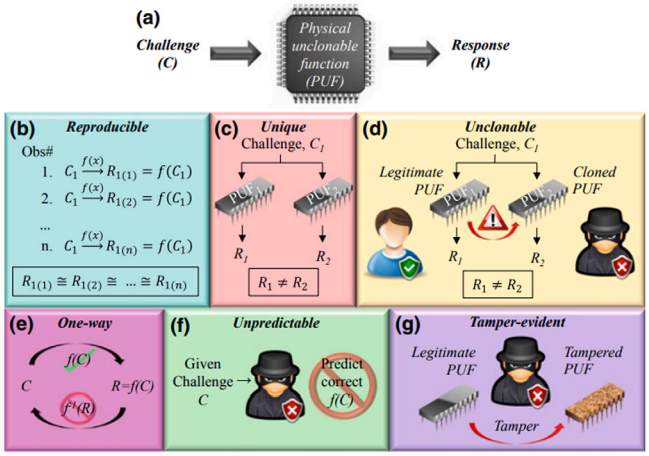

Fig. 1. Schematic representations of essential features of PUF [35].

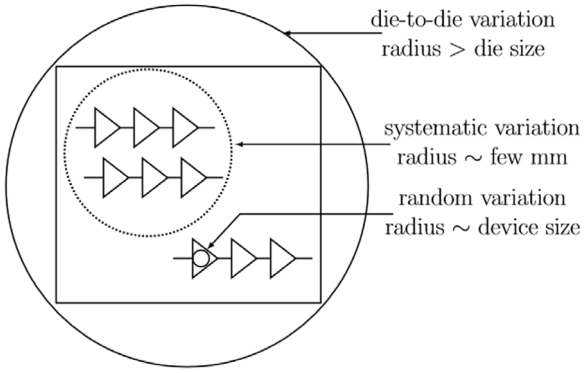

Fig. 2. Classification of silicon process variations [36].

2.1. Device variations and its impact on PUF design

PUFs challenge–response relationship is determined by manufacturing nanoscale process variations in the logic and interconnect of an integrated circuit (IC) chip. From circuit design perspective, the process variations can be generally divided into two major groups: *die-to-die variations* and *within-die variations* [36].

As shown in Fig. 2, *die-to-die variations* have a variation radius larger than the die size including within wafer, lot to lot, wafer to wafer, and fab to fab variations. These variations affect all the circuits within the die equally. On the other hand, *within-die variations* refer to the variations that occur between various circuit elements of the same die. They can be further divided into *systematic* and *random* variations. The radius of *systematic* variation is of the order of few millimeters. Moreover, the *systematic* variations are the component of variation which is caused by imperfections in the fabrication process and they normally show spatial correlation behavior. Finally, *random* variations are non-systematic which is caused by sources such as random concentration of dopants in transistors and variation in gate oxide thickness. These are intrinsic to the silicon material and cannot be controlled. Moreover, the radius of this variation is comparable to the sizes of individual devices, so each device can vary independently. The random variation is the main cause for producing random responses in a PUF which is unpredictable and unclonable. Furthermore, functionality of a PUF can be broadly divided into two parts: namely, enrollment and evaluation. During the enrollment process, a PUF is characterized to extract a reference response R_{ref} corresponding to a given challenge C. The CRP is stored in a secure database by an authorized entity. After the enrollment process is over, the PUF is deployed in applications. For

example, it may serve the purpose of device authentication or may be used as a secret key in a cryptographic operation. During evaluation, the PUF is provided with the challenge and generates a corresponding response R. If $R_{ref} = R$, a device is authenticated successfully or an encryption/decryption occurs.

2.2. Classification of PUFs

PUFs can be classified based on fabrication method and security strength, as shown in Fig. 3

2.2.1. Based on fabrication

PUFs are generally categorized in two major groups based on the fabrication [37]: Silicon and Non-Silicon PUFs.

- *Non-Silicon PUFs*: Generally these type of PUFs can be constructed using a non-silicon material such as optical, paper, acoustic and magnetic. Some examples of this type of PUFs are Optical PUF [8], paper PUF, acoustical PUF [37], magnetic PUF, etc. For more details one may refer to [37,38].
- *Silicon PUFs*: These PUFs utilize the uncontrollable manufacturing variations to generate a unique signature for each IC. According to the different source of variation, silicon PUFs can be mainly categorized as delay-based PUFs and memory-based PUFs.
  - *Delay based PUFs*: These PUFs uses differences in wiring delays within the circuit. Some examples of these type of PUFs are Arbiter PUF (exploit race conditions in ICs) [11], RO PUF (frequency variations found in ICs) [11]), etc. For more details one may refer to [38].
  - *Memory-based PUFs*: These PUFs exploit the instability of volatile memory cells. Some examples of these type of PUFs are SRAM PUF [15], RS Latch-based PUF [39], Flip Flop PUF [40], Butterfly PUF [41], DRAM PUF [42], etc. For more details one may refer to [38].

  Majority of the above silicon PUF implementations require dedicated circuit structures. On the other hand, a separate class of relatively few PUF implementations generate response from existing on-chip structures which are primarily not designed to be used as PUF. The typical examples are processor based PUFs [43,44], Scan or Design for Testability (DFT) structure based PUF [45], Hardware Embedded delay PUF [46], Glitch PUF [47], etc. For more details one may refer to [44].

2.2.2. Based on CRPs space

PUFs can also be classified into two major classes based on the size of their challenge–response space [15,48–50]: weak PUFs and strong PUFs. The classification is based only on the size of the CRP space. In general, if the number of CRPs supported by the PUF scales exponentially with its size then these are considered strong PUFs. While linear or polynomial increase in CRPs with size typically correspond to weak PUFs. The strong PUFs are secure against emulation attacks, where an adversary attempts to store all CRPs of PUF in a memory, whereas the weak PUFs vulnerable to emulation attack. Due to its large CRP space, a memory based emulation attack is infeasible. Typical examples of strong PUFs are Arbiter PUF and Bistable Ring PUF [37]. In the weak PUFs, an adversary can evaluate the PUF with all possible challenges within a few minutes. For this reason, the responses of this class of PUFs are not allowed to leave out the device (must be kept private) [51]. The SRAM PUF, Ring Oscillator-based PUF (RO PUF) and RS LPUF are typical examples of weak PUFs [37]. Furthermore, If only considering exhaustive search type of attacks, one may suggest that a strong PUF is much more robust than a weak PUF. However, most of the primitive strong delay based PUF designs are vulnerable to modeling attack, since the CRPs are highly correlated. As a result, the strong and weak PUFs are only used as the names to distinguish two different design concepts, and there is no implication on which one is better than the other. In general, strong PUFs can be used directly for authentication without additional cryptographic hardware, whereas weak PUFs are more suited to key generation and seeding a pseudo random number generator (PRNG) applications.

2.3. PUF quality metrics

PUFs are typically evaluated from the collected CRPs of several PUF instances of the same type on a specific platform. The common performance metrics are *uniqueness*, *reliability*, *uniformity*, and *bit-aliasing* [52]. Later, we also present some other important PUF performance metrics such as estimation of entropy in Section 5.1, correlation between response bits in Section 5.2 and the impact of aging on the PUF in Section 5.3.

2.3.1. Uniqueness

It measures the variation of responses obtained from different chips for the same set of challenges. If X_i and X_j are the n-bit responses of ith and jth chips respectively for the same challenge C, then the *uniqueness* (HD_{INTER}) is expressed as the average inter-chip hamming distance (HD) among k devices given by (1),

$$Uniqueness = \frac{2}{k(k-1)} \sum_{i=1}^{k-1} \sum_{j=i+1}^{k} \frac{\mathrm{HD}(X_i, X_j)}{n} \times 100\% \tag{1}$$

where $\mathrm{HD}(X_i, X_j)$ is the Hamming distance between n bit strings X_i and X_j, and k is the number of chips (devices). The ideal value of *uniqueness* is 50%.

2.3.2. Uniformity

The uniformity metric (also called randomness by Yu et al. in [53]) determines how uniform the proportion of $0's$ and $1's$ are in the PUF response and is calculated by (2).

$$Uniformity_i = \frac{1}{n} \sum_{j=1}^{n} r_{i,j} \times 100\% \tag{2}$$

where $r_{i,j}$ is the jth bit of n-bit response of ith chip. The ideal value of *uniformity* is 50%.

2.3.3. Bit-aliasing

It happens when different chips may produce nearly identical PUF responses, which is an undesirable effect. The *bit-aliasing* (also called bias in [53]) is calculated using (3):

$$Bit\text{-}aliasing_j = \frac{1}{k} \sum_{i=1}^{k} r_{i,j} \times 100\% \tag{3}$$

where $r_{i,j}$ is the jth bit of n-bit response of ith chip and k is the number of chips. The ideal value of *bit-aliasing* is 50%.

2.3.4. Reliability/Stability/Steadiness/Repeatability

It determines how efficiently a PUF can generate the same response at different operating conditions (ambient temperatures or supply voltages) over a period of time for a given challenge. For the ith chip, the average intra-chip HD is estimated using (4). Then the *reliability* of a PUF is defined by (5). Here, X_i is the reference response of the ith chip, $X_{i,t}$ is the response generated by it at time t, and s is the number of responses of same set of challenges. Note that the intra-chip HD (4) can be also used to calculate the Bit Error Rate (BER).

$$HD_{INTRAi} = \frac{1}{s} \sum_{t=1}^{s} \frac{\mathrm{HD}(X_i, X_{i,t})}{n} \times 100\% \tag{4}$$

$$Reliability_i = 100\% - HD_{INTRAi} \tag{5}$$

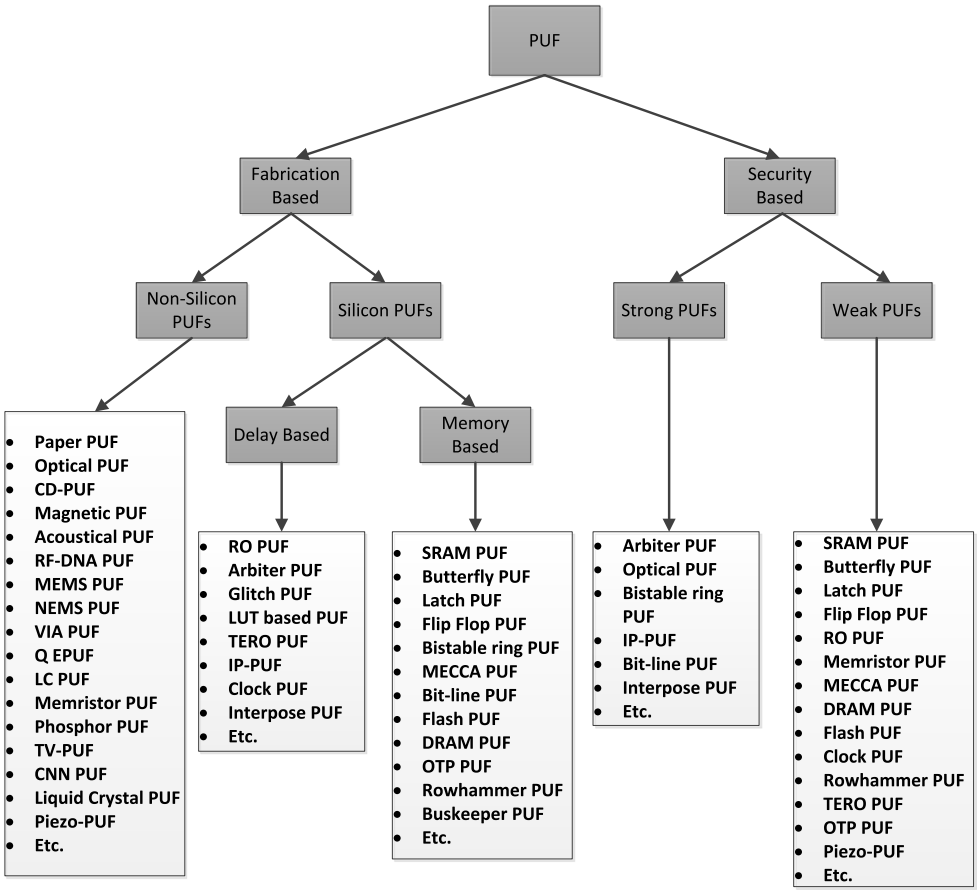

Fig. 3. Different types of PUFs.

The ideal value of *reliability* is 100% (i.e., ideal value for HD_{INTRA} is 0%) and the *average reliability* of k chips can be calculated using (6).

$$Average\ Reliability = \frac{1}{k}\sum_{i=1}^{k} Reliability_i \tag{6}$$

The evaluation metric of PUF structures may vary in different application scenarios [54]. For example, the metrics of uniqueness, reliability, and uniformity may have different importance in different PUF usage scenarios such as identification, authentication or encryption as explained below:

- Identification: The PUF can be used to generate a serial number to identify and/or track parts through manufacturing. Uniqueness is the most important metric in this situation. Further, reliability is not a major concern in the scenario of identification as long as the Bit Error Rate (BER) is relatively small. A large BER may lead to unacceptably high probability of uniqueness which reduces the number of unique IDs that can be generated and used.
- Encryption: The PUF is used to generate key for symmetric encryption algorithms and to generate a random nonce that can be used to select specific public–private key pair for asymmetric encryption. The randomness, reliability, and uniqueness are critical in such applications. However, a large CRP space is not necessary in cases where only a few keys need to be generated over the lifetime of the chip. In this case, BER must be zero which may require error correction.
- Authentication: The PUF is used to securely identify the chip in which it is embedded to an authority through corroborative evidence. In this scenario, randomness and reliability are the most critical metrics. In addition, the PUF should also have good uniqueness properties. Moreover, a very large CRP space is necessary to prevent adversaries from reading all the responses and building a clone. The large CRP space also prevents ML attacks against the SCA adversaries.

2.4. List of review/survey papers

A summary of surveys and reviews of different PUF related research areas are given in Table 1. The surveyed papers [34,37,48,55–61] have presented several typical PUFs and their applications as security primitives. McGrath et al. in [38] gave a PUF taxonomy and summarized the operation of different PUFs including all electronic and hybrid PUFs. Authors of [62] have analyzed the various lightweight authentication protocols. A survey on different PUF primitives based on emerging nanoelectronic devices are given in [63,64]. The systematic definitions of PUF performance metrics are discussed in [52,65]. Authors of [59] provided an overview of DRAM-Based Security Primitives. Delvaux et al. in [62] have discussed different helper data algorithms to enable stable PUF response regeneration. Authors of [66] an analyzed on various defense mechanisms against ML based modeling attacks on variants of strong PUFs. A brief survey on protocol attacks on advanced PUF protocols and countermeasures are given in [56]. Besides the above survey works, FPGA based PUFs have not been fully discussed and only few FPGA based PUFs are discussed in [34,37,58]. Therefore, we limit our survey to FPGA based PUFs in this paper and give a broader and more detailed survey of FPGA based PUF architectures.

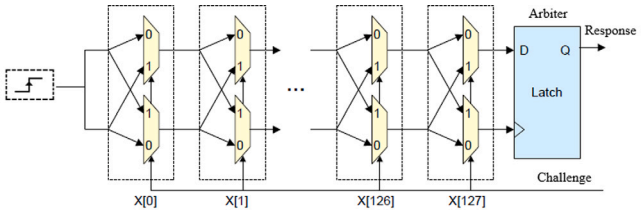

Fig. 4. Schematic diagram of Arbiter PUF.

Table 1
List of Review/Survey papers of different PUFs related research areas.

Reference	Covered different PUF related research areas
[34,37,48,55,57, 58,60,61]	An overview of different PUFs and their applications
[38]	A PUF Taxonomy
[67]	An analysis on defense mechanisms against ML based modeling attacks on strong PUFs
[68]	A survey of security attacks on weak PUF architectures
[66]	Overview and analysis of different helper data algorithms to enable stable PUF response
[69]	Physical Security in the Post-quantum Era: A Survey on SCA, Random Number Generators, and PUFs
[59]	An overview of DRAM-based security primitives
[52,65]	Detailed systematic definitions of PUF performance metrics
[62]	A Survey on Lightweight Entity Authentication with Strong PUFs
[56]	Protocol attacks on advanced PUF protocols and countermeasures
[63,64]	A survey on emerging nanotechnology-based PUFs

3. FPGA based PUF designs

The FPGA-based PUF designs can be categorized in delay-based FPGA PUFs, memory-based FPGA PUFs and FPGA based combined PUFs.

3.1. Delay-based FPGA PUFs

These PUFs use differences in wiring delays within the FPGA that are the result of manufacturing variations. There exist many delay-based FPGA PUFs and the most commonly adopted among them are the Arbiter PUF [70], RO PUF [70], TERO PUF [71], Anderson PUF [72], Pseudo-LFSR PUF [73], HELP PUF [46], and Logically Reconfigurable PUF [74].

3.1.1. Arbiter PUF (APUF)

An Arbiter PUF design proposed by Lee et al. [70], is composed of two identically configured delay paths which are stimulated by an activating signal. The arbiter (edge triggered flip-flop) determines which rising edge arrives first according to the propagation delay of the signal in the two delay paths and sets its output to 0 or 1 depending on the winner. Ideally, delay difference between these two delay paths should be 0, but it does not happen, due to random variation in delay paths, one of them is actually faster than the other and this generated response bit is random. Moreover, the challenge is used to determine the two paths that are used dynamically. For example, the circuit takes 128 challenge bits (X_i) as an input to configure the delay paths and generate a 1-bit response as an output (see Fig. 4). The individual bits of the challenge (input) are used as the select bits to the series of interconnected multiplexers and the each challenge input leads to a different path which in turn leads to a different outcome.

Arbiter PUFs come under the strong PUFs category. The last decade has seen several reports of FPGA based Arbiter PUF designs and implementations [65,75–80]. In [65], the authors presented in detail the definitions of the performance metrics and statistical evaluation results of the FPGA based arbiter PUFs. The authors of [77] have analyzed how the peculiarities of FPGA routing affect the implementations of Arbiter PUF. To improve the PUF performance metrics, Majzoobi et al. [75] introduced an arbiter PUF using programmable delay line (PDL) implemented by lookup tables (LUTs). In their design, the PUF responses are determined by applying 1000 random challenges to the 16 PUF instances. The PUF response is considered 1, if more than half of the responses are 1's, and 0 otherwise. In [78], the authors have described a design methodology to implement bias-free PDL based arbiter PUF utilizing the hard macro feature of Xilinx CAD tools. In [79], the authors also constructed PDL based compact arbiter PUF, but the PUF response is derived from the difference in counted 1's between the selected pairs of PUF instances from 32 PUF instances by applying rising clock edges. In [80], authors introduced a multiplexer-based composition of APUFs (MPUF) to enhance reliability and security properties. Most recently, a Flip-flop based APUF (FF-APUF) on Xilinx Artix-7 FPGA was reported in [76] to improve uniqueness and entropy characteristics.

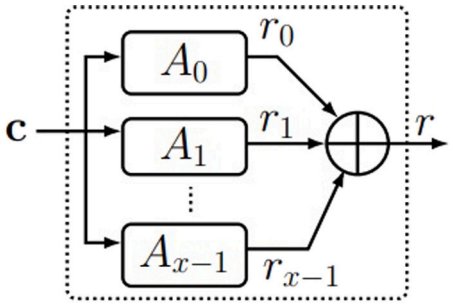

Fig. 5. Schematic diagram of XOR-Arbiter PUF.

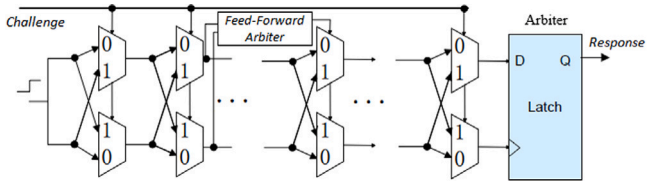

Fig. 6. Schematic diagram of Feed-forward Arbiter PUF.

3.1.2. XOR Arbiter PUF

The XOR Arbiter PUF [11] combines several rows of the basic arbiter PUFs by XORing their output into one single response bit (see Fig. 5). This makes the circuit more resistant to modeling attacks. However, this approach also introduces more errors in PUF response if the number of rows are increased.

3.1.3. Feed-Forward Arbiter PUF

It was proposed by Lee et al. in 2005 [70]. These PUFs introduce non-linearity into the signal paths by using the output of intermediate arbiters (see Fig. 6) to configure the signal paths in addition to the PUF challenge. This aids in mitigating any attack on the basic model building blocks. However, this approach increases the metastability and thus introduces more errors in PUF response. Recently, Avvaru et al. [81] provided a comprehensive performance analysis of feed-forward arbiter PUFs and feed-forward XOR PUFs (FFXORPUF) on Xilinx Artix-7 FPGAs.

3.1.4. Reconfigurable PUFs

The architecture of a secure reconfigurable PUF was introduced by Majzoobi et al. in 2009 [82]. As shown in Fig. 7, the design introduces an input network that permutes the main challenge bits using XOR

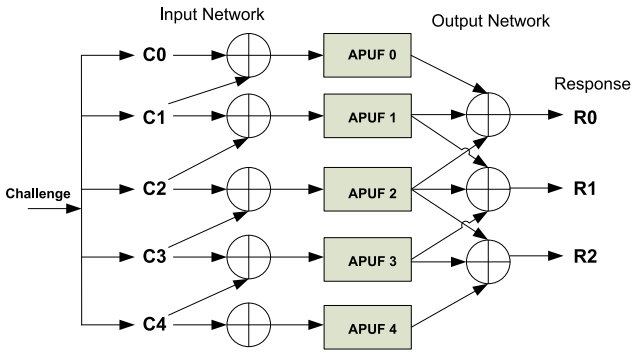

Fig. 7. Schematic diagram of Lightweight Secure.

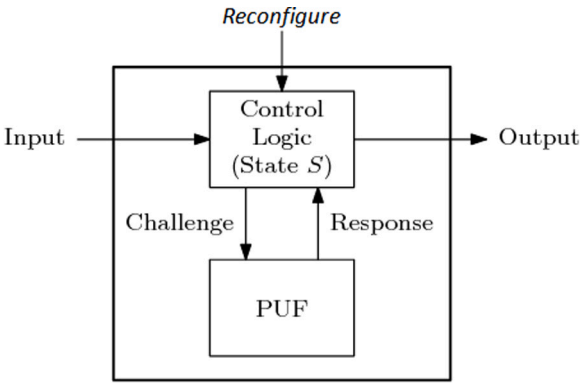

Fig. 8. Schematic diagram of Logically Reconfigurable PUF [74].

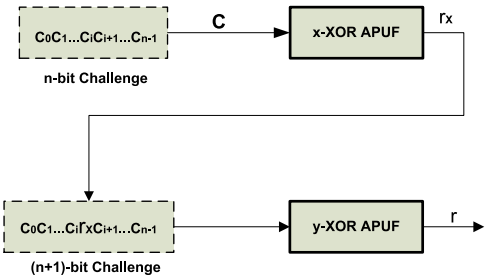

Fig. 9. Schematic diagram of Interpose PUF [84].

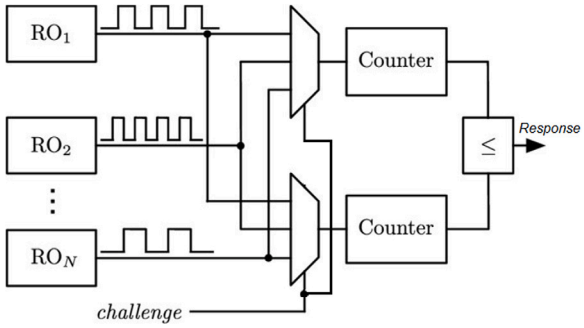

Fig. 10. Schematic diagram of RO PUF.

gates to generate the different challenge bits for each of the underlying basic arbiter PUFs. An output network is used to generate multiple bit response by combining the response bits of several basic arbiter PUFs using XOR gates. For more details, the reader can refer to the author's original work in [82].

The Logically Reconfigurable PUF (LR PUF) was proposed by Katzenbeisser et al. in 2011 [74] that combines a conventional physically unclonable function and a control logic circuit as shown in Fig. 8. The challenge/response behavior of this PUF depends on both the physical properties of the PUF and the logical state maintained by a control logic. Moreover, the LR PUF can be dynamically reconfigured after deployment by updating its state such that their challenge/response behavior changes in a random manner.

3.1.5. Interpose PUF

In 2019, Nguyen et al. [83] proposed a Interpose PUF (IPUF) on FPGAs to the repertoire of robust PUFs. It consists of two layers, the upper layer, an n-bit challenge x-XOR Arbiter PUF, and the lower layer, an (n + 1)-bit challenge y-XOR Arbiter PU. Here, the response bit of the x-XOR Arbiter PUF is interposed in between two challenge bits of the y-XOR Arbiter PUF to create a new n + 1 bit challenge. The final response bit is generated from the lower layer y-XOR Arbiter PUF with respect to the new challenge bit. Fig. 9 shows the structure of the (x, y)-iPUF. Moreover, the authors claimed that the IPUF structure is robust against machine learning based modeling attack, reliability based modeling attack and cryptanalytic attacks.

3.1.6. RO PUF

The first concept of RO PUF was proposed by Gassend et al. [9], based on a single configurable oscillator. In 2007, Suh and Devadas [11] proposed an improved RO PUF structure (see Fig. 10) which uses a number of fixed oscillators and considers the relative frequencies of oscillator pairs instead of their absolute values. The RO PUF response is derived from the difference in oscillator frequencies of selected pairs of identically designed ring-oscillators (ROs), where each RO consists of an odd number of inverters. Due to random process variation, there will be a slight difference in the oscillation frequencies even if each RO has the same structure. The frequency measurement and comparison are accomplished by using digital counters and comparators. The process is as follows: first counters count the number of edges of the two oscillating signals over a period of time and the results of the counters are sent to the comparator. Then, the comparison of two frequency counter values generate a response bit 0 or 1 for this RO pair depending on which counter had the higher value.

Note that the RO PUF is a weak PUFs [25] category, since there are a limited number of challenge bits that can configure the RO PUF's operation. Once fabricated, the RO's frequency is set based on the challenge bits, so the output bits of the PUF will always remain constant. Here, the change in input challenge leads to a different frequencies which in turn leads to a different outcome. Although, several works have reported FPGA implementations of RO PUFs in literature [10,21,25,26,79,85–96]. Merli et al. [92] demonstrated RO PUF quality improvements by adjusting RO runtime and usage of an enable and disable logic module. Moreover, they also showed a RO PUF key generation system on an FPGA using this design strategy. Furthermore, Maes et al. [10] proposed a cryptographic key generator based on a RO PUF which generates a response based on the frequency ordering of a set of oscillators. Similarly, Günlü et al. [21] proposed a method based on scalar quantizers methods to extract secret keys from RO PUF. The authors of [96] proposed an RO PUF using the initial waveform of the RO output just after the oscillation starts. Note that the reliability of FPGA based RO PUF is significantly improved by using different methods: biased placement of RO's [91], frequency offset method [25] and phase calibration process [93].

3.1.7. Configurable RO PUF

The notion of configurability in RO PUF (CRO PUF) has been introduced by Maiti and Schaumont [86] to reduce noise in PUF responses. As shown in Fig. 11, the idea is that a multiplexer is added after each inverter to select one out of two inverters at each stage of the RO.

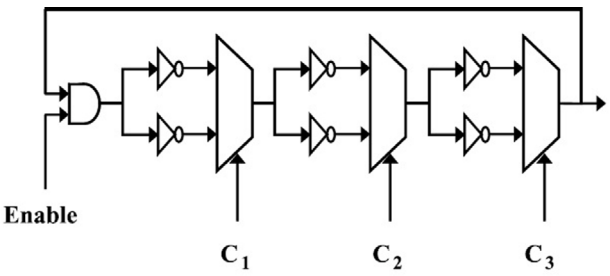

Fig. 11. Schematic diagram of configurable RO [86].

This method uses the configurations with the largest delay difference to improve the RO PUF reliability.

Authors of [87] introduced an improvement of Maiti's configurable RO PUF [86] and analyzed its performance in generating more output bits while using the same amount of area on Xilinx FPGAs. Similar CRO PUF implementations and performance improvements have been reported in [89,95]. Sahoo et al. [97] proposed current controlled configurable RO PUF in which inverters of RO uses different logic styles. Gao et al. [94] proposed another highly flexible configurable RO PUF. The basic idea is to build RO PUF at inverter level instead of RO level to improve the reliability of RO PUFs. However, to improve reliability, these techniques incur high hardware overheads. In [98], authors proposed a crossover RO PUF to improve flexibility and reliability and also to reduce hardware overheads. The key idea is to select every inverter from multiple ROs pairs with Lookup Tables (LUTs). Moreover, Habib et al. [85] and Anandakumar et al. [79] constructed programmable delay configurable based compact RO PUF on FPGAs. Furthermore, the authors of [88] introduced Loop PUF (or single RO PUF) based on controllable delay elements. The Loop PUF compares multiple elements sequentially instead of parallel during generation of the response bit. Liu et al. [26] proposed a XRRO (XOR-based Reconfigurable RO) PUF which is evolution of RO PUF that uses XOR gates instead of inverters. In 2018, Srinivasu et al. [99] proposed a configurable LFSR-based PUF (CoLPUF) that uses the LFSR states to generate different control signals for selecting the response bits from the ring oscillator frequencies. In 2018, Cui et al. [100], proposed a CRO PUF based on the tristate inverters. In that work tristate inverters were utilized as the configurable components instead of the inverters in the RO PUF and the design achieved good flexibility with low hardware cost. Recently, the authors of [101] introduced a programmable RO PUF based on the switch matrix of an FPGA which can be programmed as a chained RO PUF. This PUF design achieves good uniqueness and reliability metrics as well as a high hardware efficiency.

3.1.8. TERO PUF

The Transient Effect Ring Oscillator (TERO) PUFs is a delay based PUF which was proposed by Bossuet et al. in 2014 [71]. The TERO PUF is similar to the RO PUF, but it uses TERO cells that have two states such as a transient oscillating state and a stable state [71]. The basic structure of a TERO cell (see Fig. 12) is similar to RS flip flop. The circuit starts oscillation for a short period of time by setting the init signal to 1. Depending on the mismatch in the delays between the two branches of the TERO cell caused by CMOS process variations and that can be used as PUF response. In [24], the authors presented a comprehensive case study of the TERO PUF on Xilinx and Altera FPGAs. This TERO PUF response is derived from the difference in oscillator frequencies of selected pairs of identically designed 256 TERO cells.

3.1.9. Bistable Ring PUF

The Bistable Ring PUF (BR PUF) [102] is also similar to the RO PUF in terms of having inverter rings, but holding a stable state for progressing time. A RO PUF is composed of an odd number of inverter

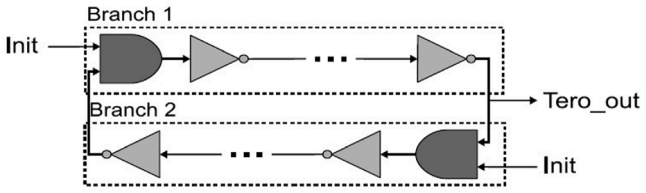

Fig. 12. Schematic diagram of TERO PUF [24].

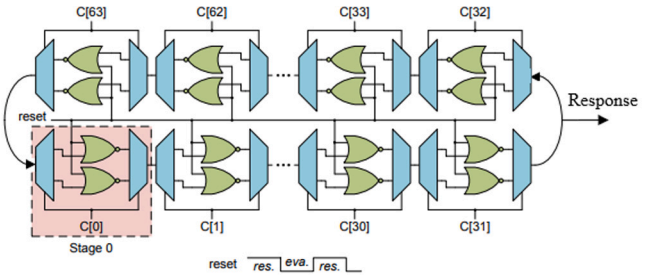

Fig. 13. Schematic diagram of BR PUF.

gates. But the BR PUF circuit consists of an even number of inverter gates, forming two types of possible states of either "101010..." or "010101..." instead of an oscillating state. The schematic diagram of one such PUF is shown in Fig. 13. Each challenge bit controls the multiplexer and de-multiplexer of this circuit and thereby defines signal delay path to take. A reset signal (see Fig. 13) is applied to enforce an initial unstable state of the ring. Responses are generated as in the BR PUF from the two possible settling states of a bistable ring after releasing the reset signal. Recently, XOR gate based reconfigurable bistable ring PUF (XRBR PUF) was proposed by Liu et al. [26]. The XRBR PUF consists of eight XOR gates where one input of the 2-input XOR is used for gate connection purpose while another is used for the reconfiguration data. Each XRBR generates a 1-bit output response. In the case of a 128-bit response, 128 XRBR PUF structure is needed.

3.1.10. Anderson PUF

It was proposed by Anderson et al. in 2010 [72] and is specially designed for FPGA, because It does not require hard macros to control symmetry as like other many PUFs designed for FPGAs. This PUF relies on the delays introduced by the LUTs (which is configured as shift-register) and the multiplexers (carry-chain logic). The schematic diagram of Anderson PUF as shown in Fig. 14. It uses the presence or absence of glitches along the carry chain to determine the output bit. This PUF has the advantage to be completely described in HDL. For more details one may refer to [72]. Most recently, Usmani et al. [12] presented enhanced implementation of the Anderson PUF on a Virtex-7 Zynq-7020 architecture that uses the more plentiful SLICELs instead of SLICEMs. The response bias of the PUF design is minimized by tuning the carry chain length, mismatch in delay of feedback paths, and glitch filtering stages.

3.1.11. HELP PUF

The Hardware Embedded Delay (HELP) PUF design based on delay measurement of complex combinational paths is an Advance Encryption Standard (AES) round operation [46]. A bitstring is generated by comparing pairs of these path delays of the AES logic core. Moreover, this PUF utilizes scan-chain flip-flops of the Xilinx FPGA and is a very high-cost implementation because it utilizes the paths delays of an AES implementation and several post-processing techniques in hardware.

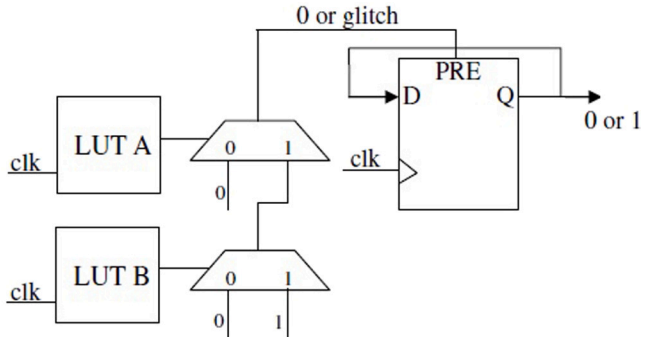

Fig. 14. Schematic diagram of Anderson PUF.

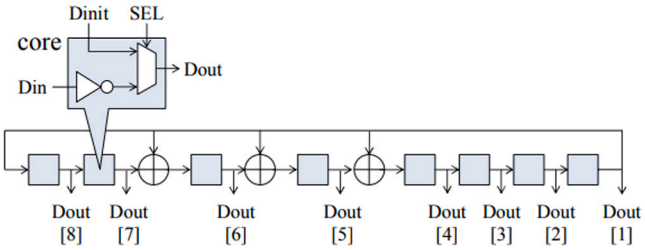

Fig. 15. Schematic diagram of PLFSR PUF [73].

3.1.12. Glitch PUF

It was proposed by Suzuki et al. [47] in 2010. The glitch PUF architecture exploits glitches that behave non-linearly from delay variation between gates and pulse propagation of each gate. The glitches in combinational circuits can be characterized by a delay line to produce unique and random bits. For more details one may refer to [47].

3.1.13. PLFSR PUF

The structure of the Pseudo-LFSR PUF (PLFSR PUF) [73] is based on the Linear Feedback Shift Register (LFSR) but it actually composes large combinational logic instead of shift register. The schematic diagram of PLFSR PUF is shown in Fig. 15. The PLPUF efficiently outputs a long-bit, produces a variable ID, and the size of the PL PUF circuit is reasonably small. Furthermore, authors of [103] evaluated the performance of MUX and LUT based PLFSR PUF implementation methods and summarized the strengths and limitations of such a PUF.

3.2. Memory-based FPGA PUFs

Memory-based PUFs exploit the power-up behavior of volatile memory cells to generate a PUF response. Contrary to the above mentioned delay-based PUFs, memory PUFs do not require specific design to generate a PUF response, but they work with standard memory element such as SRAM cells, latches and flip-flops. All these memory cells are inherently unstable circuits (bistable circuits) that can enter one of two different stable states (logical 0 or 1) by applying an external data signal input to store one bit of information. The resulting state depends on the manufacturing process variations and the noise in the circuit. Memory-based FPGA PUFs mainly include SRAM PUF [15], Butterfly PUF [41], Flip-Flop PUF [104] and RS Latch-based PUF [39].

3.2.1. SRAM PUF

The static random-access memory (SRAM) is volatile memory technology that loses its content shortly after power down. The use of SRAM as a PUF have been simultaneously and independently proposed by Guajardo et al. [15] and Holcomb et al. [105]. These PUFs use the randomness in the power-up behavior of standard SRAM memory in digital chips. The SRAM consists of array of memory cells that form

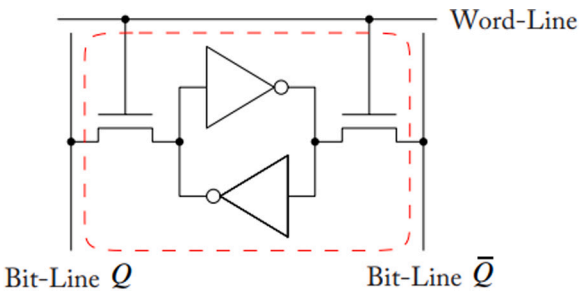

Fig. 16. Schematic diagram of SRAM PUF.

the SRAM block and each cell capable of storing one binary digit. An SRAM cell as shown in Fig. 16 is logically constructed as two cross-coupled inverters and two access transistors that used to read and write the data.

When an SRAM powers up, the initial value of each single SRAM cell can be either zero or one every time, resulting from random differences in the threshold voltages. This randomness is expressed in the startup values of uninitialized SRAM memory and an SRAM response achieves unique random pattern of 0's and 1's. Therefore, the power-up pattern of SRAM memory can be treated as a PUF where the address of each cell is a challenge and power-up state (start-up values) the response. The properties of SRAM-based PUFs were tested on FPGAs (with dedicated SRAM block) [15]. However, it turns out that in general this is not possible, since on the most common FPGAs, all SRAM cells are hard reseted to zero directly after power-up and hence all randomness is lost. Another inconvenience of SRAM PUFs is that a device power-up is required to enable the every response generation, which might not always be possible. Another SRAM-based PUF is presented by Güneysu which to utilize the write collision mismatches on true-dual port SRAMs [106].

3.2.2. Flip-Flop PUF (FF PUF)

Equivalently to the SRAM PUFs, Maes et al. proposed to read the startup values of regular flip-flops (FFs) on FPGAs in 2008 [104]. Similar to SRAM cells, uninitialized flip-flops are in an undefined state that is determined by manufacturing process variations and that can be used as PUF response. Flip-flop PUFs are based on the fact that we can prevent the initialization of the power-up values of flip-flops on Xilinx FPGAs. However, device power-up operation is required for the generation of every response. Although Gu et al. [49] proposed an efficient, scalable PUF ID generator design on Xilinx Spartan-6 FPGA that requires two cross-coupled NAND gates, two D flip-flops and one multiplexer to generate the single response bit.

3.2.3. Butterfly PUF

The Butterfly PUF [41] was proposed by Kumar et al. in 2008 (Fig. 17) that consists of a pair of cross-coupled latches which tries to mimic the startup behavior of cross coupled inverters in a SRAM cell. In other words, the inverter in SRAM PUF is replaced by latch or flip-flop to build a Butterfly PUF (see Fig. 17). By using clear (CLR) and preset (SET) functionality, a random state will be generated depending on the physical (delay) mismatch between the latches and the cross-coupling interconnect. In the case of a SRAM cell a power-up is required to generate a response, which similarly is not necessary with the butterfly PUF cell. By comparing with SRAM PUF, Butterfly PUF is suitable for FPGAs of most categories however this PUF requires carefully designed/placed in the exact placing and routing of the butterfly-cells. Moreover, the authors of [77] have analyzed how the peculiarities of FPGA routing affect the implementations of Butterfly PUF.

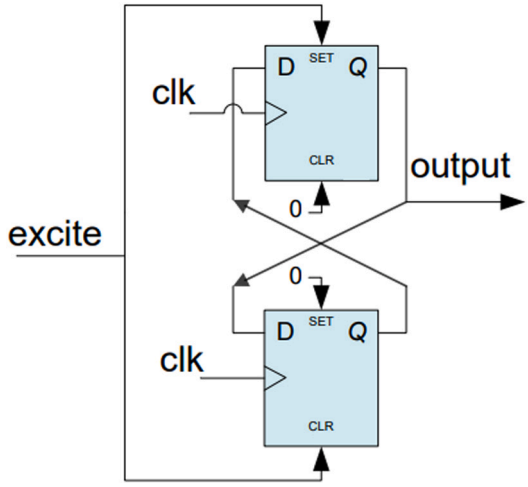

Fig. 17. Schematic diagram of Butterfly PUF.

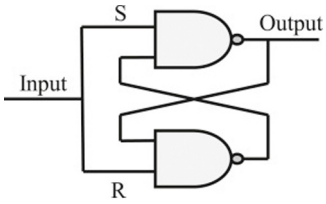

Fig. 18. Schematic diagram of SR-Latch.

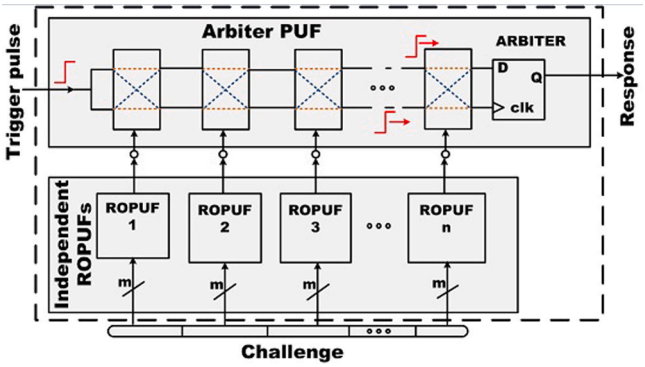

Fig. 19. Schematic diagram of Composite PUF [113].

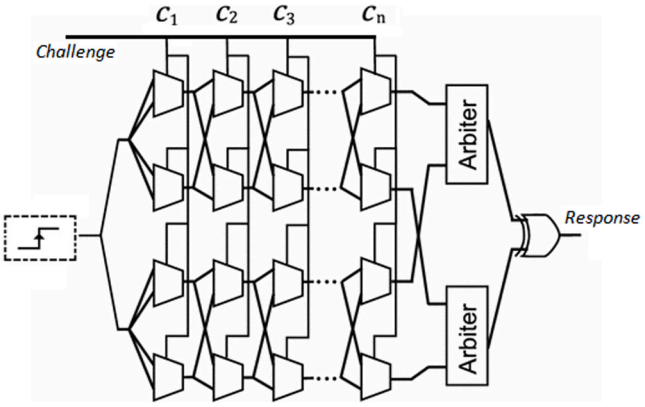

Fig. 20. Schematic diagram of 2-1 DAPUF.

3.2.4. Latch PUF (LPUF)

Latch PUF is very similar to Butterfly PUF proposed by Su et al. in 2008 [107]. This PUF generates each response using a metastable value of a latch composed of cross-coupled logic gates (i.e., NOR/NAND gates). Depending on the internal mismatch, it converges to a stable state. The recently reported RS Latch based PUFs [39,79,108,109] utilize Spartan-6 FPGA. A single SR-Latch as shown in Fig. 18 is logically constructed as two cross-coupled NAND gates. In [39], PUF response determines the final state of the output patterns (all zeros, all ones, or a combination of zeros and ones) of each latch (out of the 128 RS latches) by applying consecutive rising edges. The design in [109] determines the PUF response using the exact number of oscillations at the output of each latch (out of the 512 RS latches) during the metastable state by applying rising-edge at a control signal. In [79], the PUF response is derived from the difference in counting the numbers of 1's of the selected pairs of RS latches (out of the 32 RS latches) by applying consecutive rising edges. In [108], the design combines the feature of LUT programmable delay lines to increase the reliability of the response bits. Furthermore, Stanciu et al. [110] presented D Latch based PUFs on Xilinx Spartan-6 FPGA without programmable delay lines.

3.3. FPGA based combined PUFs

Many PUF primitives can be combined in order to enhance the PUF performance metrics and security such as hybrid and composite PUFs.

3.3.1. Hybrid PUF

The objective of hybrid PUF design is to combine two different sources of randomness (i.e., smaller PUF units) in a PUF to improve uniqueness and randomness of the responses. In [111], the author evaluates the performance of two types of hybrid PUF designs. In the first one is RO PUF and Anderson PUF are combined with each other and in the second one is the SR Latch PUF is combined with RO PUF. For more details one may refer to [111]. Most recently, area efficient design of hybrid PUF on FPGA was reported in [112]. It combines units of conventional RS Latch PUF and Arbiter PUF to improve uniqueness and entropy characteristics.

3.3.2. Composite PUF (CPUF)

The composite PUF was proposed by Sahoo et al. in 2011 [113] to achieve better quality metrics by using RO and arbiter PUFs as shown in Fig. 19. The applied master challenge is first divided into m-bit sub challenges. Then, each part of sub-challenge is applied to n independent RO PUFs. Output of the ROPUFs together form the internal challenge for the APUF that eventually generates final response. Here, the path switching activities of APUF is controlled by the responses of the RO PUFs. Moreover, authors of [114] evaluates the performance of several types of composite PUF designs.

3.3.3. Double arbiter PUF (DAPUF)

In 2015, Machida et al. [115] proposed a DAPUF as a technique to increase the uniqueness of Arbiter PUF on FPGAs where the place and route are tightly constrained. In the DAPUF, difference of the wiring bias can be canceled out when compared to the original Arbiter PUF. Moreover, the authors have analyzed different types double arbiter PUF variants in that paper namely, 2–1, 3–1, and 4–1 DAPUFs have two, three, and four duplicated APUFs, respectively. For example, a 2–1 DAPUF as shown in Fig. 20. Note that each type of DAPUF generates a response by XORing multiple one-bit outputs.

4. Attacks against FPGA based PUF designs

The FPGA based PUFs have received a lot of attention and they have been adopted by industry for many hardware-oriented cryptographic applications [49]. However, several attacks have been reported to

break the PUF security successfully [68]. They could exploit either the working model of a PUF circuit or post-processing algorithms. In what follows, we will discuss these attacks and the countermeasures in detail.

4.1. Modeling attacks

Modeling attacks are the most efficient and powerful attack for strong PUFs, where the adversary builds a accurate model (clone) of the PUF and intentionally collects a large number of CRPs to train the model. An adversary who creates a well-trained model can accurately predict the responses of unknown challenges. A successful modeling attack can replace a legitimate PUF and steal sensitive information or get access to restricted resources. Ruhrmair et al. reported that arbiter PUFs are weak against machine learning attacks [116]. They further analyzed the PUFs in the context of security protocols in [117]. We note that previous works have most often utilized machine learning based model as a way to attack the many PUF circuits [83,116,118–127]. The model of a PUF instance is built based on CRPs or some side channel information by using a machine learning technique such as a Logistic Regression (LR), Support Vector Machine (SVM) and Evolution Strategies (ES) [116,121,124]. Moreover, few delay based PUF designs have been successfully broken by deep learning based modeling attacks [84,128–130]. Recently, the authors [131] proposed advanced machine learning attacks, such as approximation attacks, could effectively model XOR and multiplexer-based obfuscation methods. More recently, Tobisch et al. [132] show that the Interpose PUF is still susceptible to reliability modeling attacks. In addition to the above-mentioned attacks against PUFs as hardware primitives, [121] has reported that recent Arbiter PUF-based protocols are also susceptible to attacks employing the CMA-ES algorithm. Furthermore, few formal frameworks have been developed for mathematical analysis of machine learning attacks against PUF designs [133–135]. The authors [133] show that the Arbiter PUF, RO PUF, XOR PUF and BR PUF can be learned with a polynomial number of CRPs and in polynomial time by using the probably approximately correct (PAC) theoretic framework. In 2019, Ganji et al. [134] introduced a publicly available testing tool such as PUFmeter for assessing the robustness of PUFs to ML attacks. In 2020, Chatterjee et al. [135] proposed a CAD framework for automated assessment of the PAC-learnability of PUF designs from an architectural level.

In order to resist modeling attacks, many defense mechanisms are proposed in the literature [11,50,70,80,82,83,120,136–152]. Suh et al. [11] proposed an XOR based Arbiter PUF that improves the ML-attack resistance of PUFs because the mapping of CRPs is obfuscated with the XOR gates. However, this approach also introduces more errors in PUF response if the number of rows are increased and incur high hardware overhead. In [70], authors introduce non-linearity into the signal paths by using the output of intermediate arbiters to configure the signal paths in addition to the PUF challenge. This approach mitigates the basic model building blocks but introduces more errors in PUF response. Majzoobi et al. proposed the lightweight secure PUF [82] that derives the individual challenges from a main challenge (using XOR gates) applied to several basic Arbiter PUFs. Then multiple individual responses are XORed to produce a multibit response. This approach improves the ML-attack resistance of PUFs but use of more XOR gates on the same rows has the potential to further expose the PUF circuit. Wang et al. [142] proposed a feedback structure to XOR the PUF response to defend against the modeling attacks. In [144], input challenges are de-synchronized from output responses by using arbiter PUF and an LFSR with reconfigurable connections to make a PUF model difficult to learn. In [140], authors proposed a challenge permutation and substitution technique to increase the ML-attack resistance of Strong-PUFs. In [50], Yu et al. proposed a lockdown technique which uses a query mechanism to restrict the number of available CRPs for attackers to prevent machine learning on PUFs. In [136], a true random number generator (TRNG) is used to select any two responses from the set which is derived from the strong PUF to obfuscate CRPs with XOR operations. Other methods try to obfuscate challenges/responses by employing a TRNG [146–150] or a weak PUF [120,139,151] to increase the resistance against modeling attacks. However, most of these PUFs are vulnerable to advanced ML attacks such as CMA-ES [121]. Vijayakumar et al. [153] suggest desirable properties that designers should aim to meet when designing ML resistant Strong PUF designs. Nguyen et al. [83] proposed the Interpose PUF (IPUF) design against machine learning attacks. It essentially consists of two XOR APUFs in which the authors claimed that using the middle bit of the second XOR APUF as the interpose position provides robustness against classical ML-based modeling attack. However, it has also been successfully attacked using machine learning [122] and deep learning [84] based modeling attacks. In [152], authors proposed a framework to protect PUF against modeling attacks by reconfiguring a conventional reliable PUF to a noisy PUF (NoPUF). Recently, few more complicated PUF designs have been proposed to make them more secure against modeling attacks. These include constructing a PUF by compositions of different PUFs [141], reconfigurable interconnected PUF network [145], and utilizing chaotic behavior of mutually interacting small PUFs [137]. Furthermore, Sahoo et al. [80] proposed to obfuscate the arbiter PUF with multiplexers instead of XOR gates to achieve higher reliability and security. However, these methods can also be broken by recently proposed approximation attacks [131]. Gu et al. [138] proposed a PUF-based authentication protocol to resist modeling-attack by incorporating two PUFs in each node, specifically a genuine PUF response and a fake PUF response. The genuine PUF responses are used for authentication while the fake PUF is queried once a while to mislead the adversary who observes the response bits. This will prevent their PUF model from correctly predicting the response to the unknown challenge. Recently, Nimesh et al. [154] presented a recurrence-based PUF which uses feedback and XOR function together to significantly improve ML-attack resistance, without significant reduction in reliability. More recently, Wang et al. [143] provided a mechanism to mix inverted responses into normal responses. This mechanism prevents ML algorithms from constructing an accurate prediction model of the internal PUF.

4.2. Physical attacks

Physical attacks correspond to gate level or transistor level characterization such as delay, leakage, or some other device metrics are analyzed [57]. Tajik et al. proposed a photonic emission analysis mechanism to characterize an arbiter PUF [155]. In a semi-invasive manner, from the IC's backside, Nedospasov et al. [156] stimulate the inverters of an SRAM PUF cell with a near-infrared laser to reveal the SRAM cell contents. Likewise, Tajik et al. [157] use a laser pulse injection to iteratively disable all but one arbiter chains that constitute an XOR PUF to learn of each individual chain of the XOR PUFs. Sahoo et al. [158] have pointed out potential countermeasures to protect the variety of delay-based PUFs which are robust against high precision laser fault attacks proposed by Tajik et al. [157]. Furthermore, the invasive active attacks on control logic of the PUF circuits can be prevented. The control logic can be enclosed by the delay wires of the PUFs [51]. These wires normally introduce path delays that the PUF circuit uses to determine its response. Therefore, if invasive attacks attempt to probe the control logic then the PUF secret will be altered and damaged.

4.3. Side-channel attacks

A PUF may be attacked passively by using side-channel information such as execution time, power consumption or electromagnetic radiation emanated from a chip containing a PUF. In 2010, Karakoyunlu et al. [159] successfully attacked the software implementation of error correction of the PUF using a power measurement based attack. In 2013, Merli et al. [160] successfully attacked a RO PUF using an EM

attack that directly targeted the PUF and not the error correction. Mahmoud et al. [161] proposed to combine modeling attacks with power side-channel attacks (SCA) to better characterize PUFs. In 2014, Becker and Kumar [162] measure power traces in order to model Arbiter PUF. Moreover, the authors in the above works have pointed out potential countermeasures to their proposed attacks [160,163]. Two countermeasures have been proposed in the work [160] to prevent localized EM attacks for RO PUF using randomization of RO measurement logic and interleaved placement. Authors of [164,165] also use the SCA information of the PUF primitives for model building. Recently, Tebelmann et al. successfully demonstrated that TERO PUFs [166] and Loop PUFs [163] are vulnerable to EM-based SCA without depackaging the chip. Moreover, Tebelmann et al. [163] introduced temporal masking countermeasure to protect the Loop PUF which randomly alters the order of challenges retaining the security subject to power and EM attacks. More recently, Aghaie and Moradi [167] proposed a general technique to protect PUF implementations against attacks based on power side channel analysis.

4.4. Cloning attacks

In a cloning attack, an adversary makes the exact physical copy (or clone) of the original PUF with identical challenge–response behavior. In 2013, Helfmeier et al. [168] has demonstrated that SRAM PUFs can be physically cloned by a Focused Ion Beam (FIB) circuit. Here, the SRAM cell's power-up contents can be obtained from the emitted infrared light. In essence, creation of a physical clone of a PUF using this method requires laboratory equipment and environment and is effectively very expensive and difficult to achieve. A simple non-invasive cloning attack on SRAM PUFs using remanence decay as a side-channel is investigated in [169]. They also proposed countermeasures use of remanence decay to improve the cloning resistance of SRAM PUFs. Furthermore, the authors of [170] have proposed a defense mechanism for RO PUF by utilizing the inverters voltage sensitivity in order to enhance the resistance of the RO PUF against the cloning attack.

5. Performance evaluation and comparison

The performance evaluation in terms of uniqueness, uniformity, bit-aliasing and reliability for the several proposed PUF designs have been carried out on different FPGA process technology in the literature range from 150 nm to 28 nm. Moreover, the number of FPGA testbeds used for performance evaluation of PUFs vary significantly in the related literature. For example, these numbers range from five to ten [49,78,89,171], above ten to fifty [11,24,25], and above hundreds [18,86,172,173]. Uniqueness measures the variation of responses obtained from different FPGAs for the same set of challenges at a core supply voltage and standard room temperature. The uniqueness metric is calculated using expression (1). The parameter uniformity identifies the proportion of $0's$ and $1's$ in the PUF response. The uniformity metric is calculated using expression (2). On the other hand, bit-aliasing signifies the undesirable effect in which different chips may produce nearly identical PUF responses. The bit-aliasing metric is calculated using expression (3). Whereas the reliability of PUFs convey about its ability to generate the same response at different conditions such as temperatures variation or supply voltages variation (see the footnotes of the Table 2) over a period of time for a given challenge. The reliability metric is calculated using expression (6). We also give in Table 2 the performance comparison of existing FPGA based PUFs implementations in terms of uniqueness, uniformity, bit-aliasing, reliability, area and response bit length. Following key observations can be deduced from these tabulated results:

- It appears clearly that the PDL based Arbiter PUF approach [78,79] outperforms when compared to the other approaches in terms of uniqueness, uniformity [52,65,76,79,115]. Furthermore, the reliability of the PDL based Arbiter PUF [115] outperforms when compared to other Arbiter PUF designs. In addition, the PDL based Arbiter PUFs [76] provide higher modeling robustness when compared to other Arbiter PUFs.
- The conventional RO PUF design [11] outperforms the other RO PUF design approaches in terms of reliability. Moreover, the configurable based RO PUF design [90] outperforms other RO PUF designs in Table 2 in terms of uniqueness. Additionally, the phase calibrated RO PUF design approach [93] improves the uniformity metric when compared to similar other RO PUF designs. The standard RO PUF designs [11,24,52,87,89–92,95] are susceptible to machine and deep learning based modeling attacks if they are used as strong PUFs. On the other hand, some of the RO PUF designs [10,21,25,26,79,96] provide higher modeling robustness when compared to the other RO PUF designs.
- For delay based PUFs, the uniqueness of IPUF [83], RO PUF [90], Loop PUF [88] and HELP PUF [46] designs are very close to the ideal value (50%) and is comparable to the other delay based PUF designs. Whereas the reliability of the Arbiter PUF design [115] outperforms comparable to the other delay based PUF designs. The Loop PUF design [88] outperforms when compared to the other delay based PUF designs. In terms of area, the HELP PUF design is occupies more resource occupation of FPGA whereas RO PUF design [79] has the lowest resource occupation of FPGA, if we compare them to the other ones in the delay based PUFs. In terms of security, Loop PUF [88], FFXORPUF [81], PLFSR PUF [73], HELP PUF [46], CoLPUF [99], RO PUFs [10,21,25,26,79,96], Anderson PUF [72] and TERO PUF designs [24,71] possess higher modeling robustness when compared to other delay based PUFs designs. However, side channel attacks have been reported to successfully break some of these PUFs [160,163,166] (see in Section 4).
- For memory based PUFs, the uniqueness of SRAM PUF [83] and FF PUF [90] are very close to the ideal value (50%) and is comparable to the other delay based PUF designs. Furthermore, the PUF ID [49] design outperforms all other memory based PUF designs in terms of reliability. In terms of area, the PDL based Latch PUF design [79] occupies less resource occupation of FPGA if we compare them to the delay based PUFs. In terms of security, all memory based PUFs provide higher modeling robustness because all these PUFs fall in the category of weak PUFs. We assess model building to be impossible for these PUFs considering that the physical random elements contributing to different CRPs are independent of each other to a great extent. However, physical and cloning attacks have been reported to break some of these PUFs [15,156,168] (see in Section 4).
- For the combined PUFs, the composite PUF (CPUF) design [115] outperform when compared to other combined PUFs in terms of uniqueness whereas of the composite PUF design [112] outperform when compared to other composite and hybrid PUF designs in terms of reliability and uniformity. In terms of security, the composite and hybrid PUFs provide higher modeling robustness when compared to other combined PUFs.

Next, we also briefly present further the another important PUF performance metrics, i.e. estimation of entropy in Section 5.1, correlation between response bits in Section 5.2 and the impact of aging on the PUF in Section 5.3.

5.1. Entropy estimation

The PUF response is expected to be unpredictable such that an adversary cannot efficiently predict the response of a device through an observation of other devices which he/she may own or have access to other devices. In order to verify this unpredictability, a number of methods have been proposed for estimation of entropy of a PUF. The

Table 2
Comparison of hardware resource consumption and metrics of different FPGA based PUF designs.

Types	Design		Unique ness	Reliability	Uni formity	Bit-alias	Area (Total	Res. bit	Target	Modeling
		Ideal Value	50%	100%	50%	50%	Slices)	length	FPGA	robust
		Gu et al. [76]	41.53	95.50[c]	–	–	2816	64	Artix-7	✓
		Sahoo et al. [78]	45.25	95.93[a]	48.30	–	–	90	Spartan-3	×
	Arbiter	Maiti et al. [52]	18.37	99.76[c]	55.69	19.57	–	128	Virtex-5	×
	PUF	Machida et al. [115]	04.70	99.32[a]	54.78	–	177	128	Virtex-5	×
		Hori et al. [65]	36.75	98.48[c]	42.34	–	–	128	Virtex-5	×
		Anandakumar et al. [79]	44.30	96.00[a]	48.45	–	234	256	Spartan-6	×
	FFXORPUF	Avvaru et al. [81]	49.20	89.50[a]	50.00	–	–	64	Artix-7	✓
	IPUF	Nguyen et al. [83]	49.25	89.50[c]	55.90	–	–	64	Artix-7	~
	LR PUF	Katzenbeisser et al. [74]	–	–	–	–	425	64	Spartan-6	×
	HELP PUF	Aarestad et al. [46]	≈ 50[d]	–	–	–	3986	–	Virtex-II	✓
	PLFSR	Hori et al. [73]	31.00	–	43.00	–	–	128	Virtex-5	✓
Delay	PUF	Nozaki et al. [103]	–	62.36[a]	–	–	–	128	Virtex-5	✓
Based		Maiti et al. [52]	47.24	99.14[c]	50.56	50.56	–	511	Spartan-3E	×
PUFs		Merli et al. [92]	48.51	98.28[a]	–	–	512	128	Spartan-3E	×
		Yu et al. [89]	47.00	–	–	–	420	64	Spartan-3E	×
		Maiti et al. [86]	44.10	99.00[c]	–	–	–	256	Spartan-3	~
		Gao et al. [94]	47.31	95.95[c]	–	–	–	96	Spartan-3	~
		Xin et al. [87]	40.00	98.98[c]	–	–	–	63	Spartan-3	×
		Habib et al. [85]	48.30	97.88[a]	50.13	51.80	747	283	Spartan-3	~
		G̀unĺu et al. [21]	–	–	–	–	849	255	Zynq-7000	✓
		Suh et al. [11]	46.15	99.52[c]	–	–	–	128	Virtex-4	×
	RO PUF	Zhang et al. [25]	49.33	95.45[c]	49.50	–	186	136	Virtex-5	✓
		Maes et al. [10]	48.40	90.31[b]	–	–	952	49	Spartan-6	✓
		Tanamoto et al. [96]	32.52	96.96[c]	55.66	–	–	256	Spartan-6	✓
		Stanciu et al. [110]	58.58	94.00[b]	–	–	–	–	Spartan-6	~
		Marchand et al. [24]	55.00	94.50[c]	–	–	–	128	Spartan-6	×
		Cui et al. [90]	49.97	98.41[b]	–	–	–	–	Spartan-6	×
		Anandakumar et al. [79]	47.13	99.16[a]	50.61	–	82	256	Spartan-6	✓
		Liu et al. [26]	48.76	97.72[b]	–	–	–	64	Spartan-6	✓
		Chauhan et al. [91]	49.83	99.35[c]	–	–	–	255	Artix-7	×
		Choudhury et al. [95]	47.40	–	49.20	49.10	–	124	Artix-7	×
		Yan et al. [93]	–	99.33[a]	50.05	–	–	128	Kintex-7	✓
	Loop PUF	Cherif et al. [88]	49.94	95.00[a]	50.00	–		15	Cyclone II	✓
	CoLPUF	Srinivasu et al. [99]	49.20	99.99[d]	48.3	47.8	572	128	Artix-7	✓
	Anderson	Anderson [72]	47.65	96.40[b]	–	–	–	128	Virtex-5	✓
	PUF	Usmani et al. [12]	46.25	97.63[a]	45.88	45.88	–	128	Zynq-7000	✓
	BR PUF	Chen et al. [102]	14.80	99.20[c]	–	–	–	64	Virtex-II	×
		Liu et al. [26]	40.67	98.22[b]	–	–	–	64	Spartan-6	×
	TERO	Bossuet et al. [71]	48.00	98.30[c]	–	–	–	126	Cyclone II	✓
	PUF	Marchand et al. [24]	48.50	85.00[c]	–	–	–	128	Spartan-6	✓
	SRAM PUF	Guajardo et al. [15]	49.97	88.00[b]	–	–	–	128	–	✓
Memory	FF PUF	Maes et al. [40]	≈ 50[d]	95.00[a]	–	–	–	255	Virtex-II	✓
based	Butterfly PUF	Kumar al. [41]	43.16	96.20[b]	–	–	130	50	Virtex-5	✓
PUFs	PUF ID	Gu et al. [49]	45.60	99.42[c]	–	–	128	128	Spartan-6	✓
		Ardakani et al. [108]	49.32	98.80[c]	44.65	44.65	128	127	Spartan-3	✓
	Latch PUF	Yamamoto et al. [39]	49.00	96.34[c]	–	–	256	256	Spartan-6	✓
		Habib et al. [109]	49.24	98.87[c]	–	–	324	256	Spartan-6	✓
		Stanciu et al. [110]	34.73	92.00[b]	–	–			Spartan-6	✓
		Anandakumar et al. [79]	48.10	99.19[a]	50.20	–	54	256	Spartan-6	✓

(*continued on next page*)

context-tree weighting (CTW) algorithm is employed to estimate the upper bound of entropy (i.e. best case) [174,175]. If compression is possible, the PUF responses do not have full entropy. Min-entropy is another metric widely employed to evaluate the lower bound of entropy

Table 2 (*continued*).

Types	Design		Uniqueness	Reliability	Uniformity	Bit-alias	Area (Total Slices)	Res. bit length	Target FPGA	Modeling robust
		Ideal Value	50%	100%	50%	50%				
Combined PUFs	Hybrid PUF	Khoshroo et al. [111]	39.63	93.63[c]	48.30	25.20	–	128	Virtex-2	∼
		Anandakumar et al. [112]	49.41	99.22[c]	50.09	50.09	257	256	Spartan-6	✓
	DAPUF	Machida et al. [115]	50.24	88.20[a]	53.94	–	436	128	Virtex-5	∼
	Composite PUF	Sahoo et al. [114]	49.04	97.48[a]	50.07	–	1051	–	Spartan-3	✓

[a]Under normal operating temperature (i.e. with standard supply and at room temperature).
[b]Under temperature variation only.
[c]Under temperature and voltage variation.
[d]Required post-processing.
✓ provides much higher robustness against modeling attacks.
∼ provides medium robustness against modeling attacks.
× provides very low robustness against modeling attacks.

(i.e. worst case) [65,175]. It is the worst-case (i.e. lower bound of the unpredictability of the response) measure of uncertainty for a random variable. Furthermore, the method described in NIST specification 800-90 [176] for evaluating the min-entropy of a binary source. The n-bit responses of k devices have an occurrence probability at each bit of p_1 and p_0 for the values of 1 and 0 respectively. When $p_{i\,max}$ is the maximum value of these two probabilities (i.e., $p_{i\,max} = \max(p_1, p_0)$), the definition for min-entropy of each individual bit (binary source) is given by (7). Here, $p_{i\,max}$ is given by (8).

$$H_{min,i} = -\log_2(p_{i\,max}). \tag{7}$$

$$p_{i\,max} = \begin{cases} \frac{HW_i}{k} & \text{if } HW_i > \frac{k}{2} \\ 1 - \frac{HW_i}{k} & \text{otherwise} \end{cases} \tag{8}$$

where HW_i is counting the number of $1's$ in k devices. The total min-entropy of the design is given by (9) and is calculated by averaging the estimated min-entropy of each bit.

$$(H_{min})_{average} = \frac{1}{n}\sum_{i=1}^{n} H_{min,i} \tag{9}$$

To get meaningful entropy value of PUF designs, a very large scale analysis with many boards is used. Even though the largest conducted experiment consisted of more than 100 boards [86,172,173,177] it is not really enough to accurately determine the entropy. In general, how to determine the exact entropy of the PUF responses is another very important open research problem.

5.2. Correlation between bits

Correlation indicates whether a relationship exists between FPGA devices in which the bit response from one device can be used to predict the response from another device. If the bits are highly correlated then an attacker might be able to predict the secret from collected responses of other devices. Systematic aspects to process variation may show up as significant correlation at particular intervals. The responses are extracted from a common fabric and hence it is possible for spatial correlation to appear. To check the presence of correlation in the test chip, the auto-correlation test specified in AIS31 (T5) [178] or auto-correlation function R_{xx} (10) is used [113,179]. Here, x is the n-bit response being observed, and R_{xx} is evaluated at lag j. This value tends towards 0.5 for uncorrelated bit-strings and towards 0 or 1 for correlated bit-strings. For example, the minimum R_{xx} among 22 different 64-bit responses (i.e., 22 FPGAs) for Arbiter PUF and Flip-flop based APUF designs is 0.23 and 0.54, respectively [76]. Similarly, the minimum R_{xx} among 184 different 64-bit responses for the RO PUF is 0.73 [180].

$$R_{xx}(j) = \frac{1}{n}\sum_{i=1}^{n} x_i \oplus x_{i-j} \tag{10}$$

5.3. Aging effect

Silicon devices can be affected by various aging mechanisms including Hot Carrier Injection (HCI), Time Dependent Dielectric Breakdown (TDDB), oxide breakdown, Negative Bias Temperature Instability (NBTI), and Electro-Migration (EM) resulting in performance degradation and eventually design failure [181]. For example, to consider the aging effect on FPGA based PUFs, accelerated aging tests [181–183] were performed by using voltage and temperature stress. In [181], the authors showed the aging degradation of the ROs on Xilinx Spartan3E FPGAs by raising the temperature to 80 °C and the supply voltage to 2 V (nominal is 1.8 V) for up to 1100 h and PUF output was recorded every 1 h. This is sufficient to induce more than 10 years of aging [181]. These responses were then compared with the reference ones generated at normal temperature with standard supply voltage. The average performance degradation for 193 FPGAs is 9.5%. Similarly, the authors in [181,182] showed that the aging degradation of the ROs with the stress conditions of 125 °C and 1.8 V (nominal is 1.2 V) for up to 50 h and PUF output was recorded every 3 h. The average performance degradation for 20 FPGAs is 1.283%.

6. FPGA based PUF application scenarios

In this section, we briefly present the five classes of application scenarios which we envision for PUFs, i.e. PUF-based secret key generation in Section 6.1, PUF-based key sharing in Section 6.2, PUF-based IP protection in Section 6.3, PUF-based logic obfuscation in Section 6.4, PUF-based software protection in Section 6.5, PUF-based random number generator in Section 6.6, PUF-based identification in Section 6.7 and PUF-based authentication in Section 6.8.

6.1. PUF-based secret key generation

In many security applications, cryptographic primitives such as encryption, message authentication code and digital signatures play central roles. The security of the cryptographic protection relies on the secrecy of the key. PUF responses are inherently noisy and also sensitive to varying environmental conditions and they cannot be used as keys directly. Therefore, error correction mechanisms are necessary on-chip to generate secure and reliable cryptographic keys.

The secret key generation process essentially involves two phases. The first is the key enrollment phase whereas the other is the key regeneration phase as depicted in Fig. 21. During the key enrollment phase, a syndrome information such as helper data is calculated from the raw response of the PUF using encoding circuit of the error correction code (ECC) and the helper data is public information and can be stored anywhere (i.e., on-chip, off-chip, or remotely on a server) that allows for correcting bit-flips in regenerated PUF outputs. In the key regeneration phase, the noisy response from the PUF is taken and the helper data (for same challenge) are given to the decoder

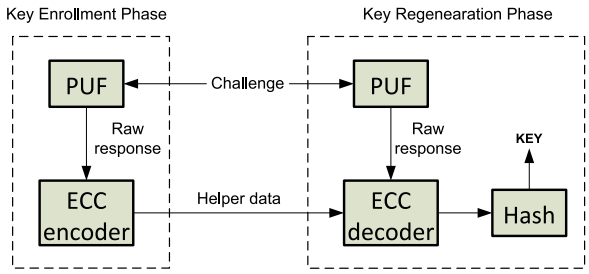

Fig. 21. PUF-based Secret Key Generation.

of the ECC. With the help of the stored helper data, the ECC will correct the errors that occurred in the response of the PUF due to any changes in the temperature and voltage. Finally, the corrected output are simply hashed down to a desired length and used as a cryptographic key. A wide variety of error correction mechanisms have been reported for PUFs on FPGAs [10,16,21,99,184–189]. The binary extended Bose–Chaudhuri–Hocquenghem (BCH) codes are particularly widely used for correcting bit flips in PUF responses due to its strong error-correcting properties [10,16,21,99,184–186]. Repetition codes have often used in concatenated coding schemes to remove the noisy bits in PUF response [188]. Index-based syndrome (IBS) coding [186] also used to remove the noisy bits in PUF response. In [187], Reed–Muller codes and the Golay code are used. In [189], a convolutional code is used for implementation in the Seesaw decoder on FPGAs. Furthermore, some practical PUF-based key generator designs have been reported on FPGAs [10,12]. In [10], authors presented a complete implementation of a PUF-based key generator based on a ring-oscillator PUF, BCH error-correcting and a cryptographic entropy accumulator. This complete implementation occupies 1162 slices on Xilinx Spartan-6 FPGA of which 82% (952 slices) are occupied by the ROPUF and only 18% by the key generation logic (i.e., error-correcting and hashing). An efficient PUF-based key generation in FPGAs using per-device configuration is presented in [12]. This per-device PUF configuration step to reduce area resources for PUFs and error correction as used in the key generation and occupies 2199 LUTs and 2689 flip-flops on Xilinx Virtex-7 FPGA.

6.2. PUF-based key sharing

Traditional PUFs generate chip-unique key for each device which cannot be cloned in another device due to manufacturing process variations. However, the shared key is required in multiparty communication between different devices in many IoT security applications. In order to address this issue, Zhang et al. [190] proposed a PUF-based key-sharing method based on the crossover configurable RO PUF. This PUF has flexible configuration of challenges with interstage crossover structures in each level of inverters of RO PUF. Therefore, with the appropriate selection of configurations and challenges, different devices can produce the same response as the shared key for resource-constrained devices. For more details one may refer to [190].

6.3. PUF-based IP protection

Another field of application for PUFs is using them in protection of brand and Intellectual Property (IP). Since it became common for the semiconductor industry to have shared foundries or outsourcing the production, counterfeiting of devices and IP is an increasing problem. As already mentioned, it is possible to derive cryptographic key material from PUFs and this in turn can be used to encrypt and decrypt IP and thereby bind it to a certain FPGA chip. Such an approach using bitstream encryption was suggested by Kean [191]. Moreover, the literature introduced a number of solutions for guaranteeing IP security based PUF in hardware-reconfigurable systems. For example, secret keys created by a PUF are used to initiate IP core operation and periodically authenticate its use [192]. An improvement of this approach can be found in [15], where secret-key material gathered from the PUF is used both for encryption and MAC-based authentication. A comprehensive approach to protecting the use of intellectual property cores in FPGAs using public/private keys pairs is described in Kumar et al. [41]. Closely related to IP protection is the topic of remote feature activation [193]. This means that a certain feature of the device may be unlocked after the device is already in possession of, an often untrusted third party. In [194], the authors introduced a protocol based on public-key cryptography and PUF for the IP protection on FPGAs. Due to the public-key cryptography for the IP protection on FPGAs, the authors observe that by the corresponding private-key does not need to ever leave the FPGA and thus increases the security of the overall system.

Roy et al. [16] proposes a two-party pay-per-use licensing scheme to addresses the issue of IP overuse on FPGA. The basic building blocks of the proposed scheme are lightweight symmetric encryption for IP protection, PUF for device binding and re-configurable look-up table (RLUT) for IP activation. In [195], authors reported a complete novel framework for hardware IP licensing and protection on FPGAs in public clouds. The authors of [196] describe enhanced authentication mechanism by using hybrid PUF with FSM for protecting IPs of SoC FPGAs. In [197], authors presented use of the reconfigurable digital PUFs as instruction-level authentication devices that offers anti-piracy protection for the software and hardware IP. Similarly, [198] presents a binding scheme that binds the hardware IPs to specific FPGAs by utilizing the PUF and the FSM of circuits. Parrilla et al. [199] describe the protection and hardware activation protocol of IP cores implemented on FPGAs by using PUFs and elliptic curve cryptography. In this scheme, 5200 LUTs are utilized for implementing the hardware activation protocol on Spartan-6 devices and the total time required to complete an activation process is around 32.6 ms.

6.4. PUF-based logic obfuscation

Logic obfuscation is an approach to prevent integrated circuit piracy and counterfeiting. The logic obfuscation attempts to hide the genuine functionality of a circuit by inserting built-in locking mechanisms into the original circuit. The locking circuit become transparent and the circuit will function correctly only when a right key is applied. In [200], the authors introduce a PUF-based logic that uses delay-based PUFs together with reconfigurable logic for the obfuscation of an arbitrary circuit. The design is functional when the reconfigurable circuits are correctly programmed, i.e. setting the PUF inputs so that the circuit functions correctly, by the designer or end-user. This approach provides a custom solution for each chip but it comes at considerable area and run-time costs. Jiliang Zhang in [201] uses the configuration of obfuscation cells of obfuscated design to XOR with the PUF response in order to generate a chip-dependent license to prevent piracy and over-building attacks and to provide the pay-per-device licensing service. In that case, an attacker cannot compute the correct license to unlock the pirated/overproduced chips because the attacker has no knowledge about the key of the obfuscation cells. Hence, the designer (i.e., who can issue the license) is the only one to activate the chip. The basic idea is when the chip is powered on, the PUF response will XOR with the license to generate the correct key bits for obfuscation cells. Then the generated key bits get stored in the flip-flops to unlock the chip.

6.5. PUF-based software protection

PUF entangled software protection scheme that leverages PUF features to protect software code from malicious modification before or during run-time. In 2010, Atallah et al. [202] utilized PUF technology to bind software to native hardware in a virtualization environment. In 2010, Gora et al. [203] proposed a complete, end-to-end design flow to

bind a software code to an FPGA utilizing a RO PUF. At the device start, they derive a 128-bit AES key from the PUF. Then utilize this key to decrypt the actual software code that was stored encrypted beforehand, and finally execute the decrypted software. As a result, developers want to ensure that their software code is protected from being exposed to or tampered with by unauthorized parties. In [204], the authors showed that SRAM PUF responses combined with self-checksum code can protect software from modification and from running on unauthorized devices. In 2016, Qiu et al. [205] presented PUF based linear encryption method against code reuse attacks to enhance software security. This method entails the PUF responses to encrypt the return address without extending the instruction set architectures (ISAs) and without modifying the compiler. However, attackers can easily infer the encryption key by obtaining an encrypted address. In 2019, Zhang et al. [206] proposed a hardware-assisted control-flow integrity (CFI) checking technique, where the PUF is used as a secret key to assist CFI verification. Here, the Hamming distance (HD) matching method is used to avoid the leaking of the key without changing the ISAs and compiler. In 2020, Wenjie et al. [207] presented a hardware-entangled software protection scheme which leverages dynamic PUFs. A dynamic PUF has time-dependent responses which is different from the usual static PUFs. In this method the dynamic PUF response and linear self-checksum functions over the prime finite field can be used to detect if a software is running/was modified on an authorized device.

6.6. PUF-based random number generator

Cryptographic applications use random numbers to generate encryption keys, create initial parameter values, random padding bits, and to generate random nonces into authentication protocols. In most cases these random numbers come from a PRNG that is a deterministic software algorithm which imitates randomness. PRNG is used to generate stream of random bits by using secret seed values. If an adversary knows the seed to a PRNG, then he/she can generate and predict the entire stream of random numbers. Hence the PRNG seeds should be unique, completely random and unpredictable in nature. The PUF responses can be used as secret seeds of PRNG and to generate a stream of true random bits. Several FPGA based implementation have been reported about using PUFs for random number generation [14,208–210]. In [14], the initial seed is generating from PUF that includes dynamic refreshing logic to ensure that the random numbers generated are nonperiodic. This design achieves 832 Mbps throughput and also consumed less resource 352 slices on the Xilinx Virtex-7 FPGA [14]. In [208–210], the true random seeds are extracted from the noise on the start-up pattern of SRAMs and given as an input to a nondeterministic random number generator. The design [208] achieves 803 Mbps throughput and also occupies 369 slices on the Xilinx Virtex-II Pro FPGA. The throughput of design [209] is up to 400 Mbps whereas design [210] achieves 598.1 Mbps on Altera Cyclone IV FPGA.

6.7. PUF-based identification

Using PUFs for device Identification is more interesting for anti-counterfeiting technologies because of their physical unclonability property. In this scenario, the PUF response can be used directly for unique identification (chip ID) very similarly as in a biometrical identification scheme. During an enrollment phase, a server queries multiple PUF devices in order to create a database of their CRPs and stores them in a database together with the identity of the physical system embedding the PUF. During identification phase, the server picks a random CRP from the CRPs stored in the database for the current device and sends challenges to the current PUF device. If the observed response is matched with the relevant responses stored in its database, the identification (ID) of the device is successful, otherwise it fails.

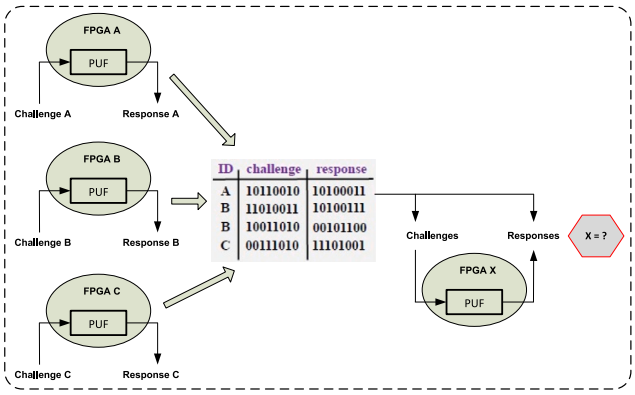

Fig. 22. PUF-based Identification.

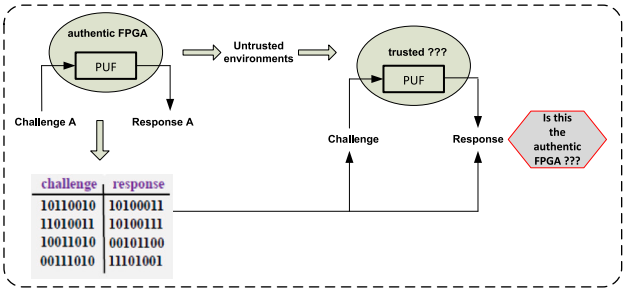

Fig. 23. PUF-based Authentication.

Fig. 22 demonstrates how PUF-based identification works. Moreover, each CRP should be used only once for every PUF and has to be deleted from the database after the identification in order to prevent replay attacks. Moreover, error correction may be required again if the PUF responses are noisy. Furthermore, some practical chip identification generator have been demonstrated on FPGAs [42,89,211]. In [89], authors presented a chip ID generation method with configurable RO, power-up initialization and adaptive re-initialization can considerably improve its repeatability. PUF based identification on Xilinx Spartan-6 FPGA devices is presented in [211] that makes use of the absolute values of ROs frequencies, which is immune to working conditions and aging. In [42] presented a DRAM-based PUF for chip identification and achieved the most stable bits to be used as a 128-bit identifier.

6.8. PUF-based authentication

PUF-based authentication is almost similar to PUF-based identification, as once again a server creates a database of the CRPs of a PUF device. However, this time the PUF device goes through untrusted environments, where it can be substituted by counterfeits. Therefore, if a client wants to ensure the authenticity of such a PUF device, it can do so by sending a request for authentication of the relevant PUF device to the server. Then, the server responds with a challenge for the PUF device to be identified. If the produced response matched with the relevant response stored on the server's database, then the device is authenticated, otherwise it is considered a fake device, as shown in Fig. 23. In this case, authentication is achieved without explicitly revealing any of the stored CRPs. Again, however, in the case of successful authentication, a CRP of the authenticated PUF may be completely revealed, and therefore should not be used again. In order to avoid this, a hashing scheme may be employed, so that the CRPs are not transmitted in the clear, or a key can be used for implicit authentication, etc. Finally, error correction may be required again if the PUF responses are noisy.

Furthermore, some practical PUF based authentication systems have been demonstrated on FPGAs [13,19,148]. In [13], authors presented an intrinsically reconfigurable DRAM PUF based on the idea of DRAM refresh pausing and also they demonstrated the use of this PUF in performing device authentication through a secure, low-overhead methodology, on Altera Stratix IV GX FPGA-based Terasic TR4-230 development board. In [19], the authors demonstrated authentication and key exchange protocol on FPGA by combining the concepts of PUF, IBE and Keyed Hash Function. This complete implementation occupies 1733 slices on Xilinx Artix- 7 FPGA of which 456 slices are occupied by the Double Arbiter PUF and 1277 slices are occupied by the BCH error-correcting logic. The authors in [212] proposed PUF-based anonymous authentication scheme on FPGAs for hardware devices and IPs in edge computing environment. Authors in [148] introduced slender PUF protocol based on pattern matching that to authenticate the responses generated from a Arbiter PUF. This PUF protocol implementation requires 652 LUTs and 1400 registers in Xilinx Virtex 5 FPGAs. In [213] authors demonstrated a FPGA implementation of a provably secure protocol that supports privacy-preserving mutual authentication between a server and a constrained device. This PUF protocol implementation requires 3543 LUTs, 1275 registers and 8 block RAMs in Xilinx Virtex 5 FPGAs.

7. Future research directions

After the overview and study of PUF properties, designs, attacks performance comparison and application scenarios, respectively in Sections 2, 3, 4, 5 and 6, we touch upon some discussion points and open questions. The field of PUFs has grown significantly in the past decade but still a lot more still needs to be done to meet the future requirements. In this section, we try to point out some interesting future research directions.

- *Entropy Estimation*: As discussed in Section 5.1, a number of works proposing PUFs provide an estimate of the upper bound of entropy or lower bound of entropy for measure of uncertainty for a random variable. A general question not solved by the PUF community yet is the exact entropy estimation of a PUF response. This requires a further study of the exact entropy estimation of a PUF response.
- *Security*: As discussed in Section 4, several attacks have been reported to break the PUF security successfully. PUF has attracted a lot of attention in recent years for PUF based IP protection. However, existing PUF based IP protection methods provides few CRPs since the PUF physical characteristic response is singular. As a consequence, illegal users may easily forge the characteristic by establishing ML based mathematical model. In this context, PUF techniques should be integrated with more secure mechanisms for secure protection of PUF based IP circuit in FPGA-based SoCs. Although many authors have pointed out potential countermeasures to their proposed attacks (see Section 4), most of these countermeasures do not have high practical value because they incur high hardware overhead while still cannot provide full security for the PUFs. Therefore, robust PUF designs resilient with respect to modeling attacks need a further study. Furthermore, If PUFs are ever to be used in high security systems then they should resist physical and side-channel attacks. So far, very little has been done in this area. Clearly, this is an area in which a lot of progress can still be made.
- *CAD Frameworks for Design and Test for PUFs*: Several design schemes for PUFs have been discussed in Section 3. However, for PUF designers and manufacturers, it still remains an open question whether their new designs or countermeasures are resistant to ML and other type of attacks. Only few CAD frameworks for design and test for PUFs have been reported in the literature (see Section 4). However, these frameworks are only supported for the robustness assessment of PUFs against ML attacks. Therefore, there is a need to develop CAD frameworks for robustness assessment of PUFs against all type of attacks.
- *Environmental Influences*: As discussed in Section 5, PUF responses are subject to unwanted physical influences from their direct environmental noise caused by the voltage and temperature variations. For a PUF to be used in a practical application, these effects should not be affected in order to prevent unforeseen failures. From this perspectives, any new FPGA based PUF designs response should not be influenced by the FPGA temperature, supply voltage, and device aging.
- *Hardware Overhead*: As discussed in Sections 3 and 6, FPGA based PUFs design and its applications should be as cost-effective as possible in order to be of any practical use. Measuring the implementation and operation cost in terms of area, speed, and power consumption is an exercise which should be made for any hardware implementation and is not limited to PUFs.

8. Conclusions

To identify the state-of-the-art in the field of PUF circuit design and implementation, we conducted and presented in this article a FPGA based PUF designs. The purpose of this article is to help readers (including practitioners and researchers) with the most comprehensive survey of the current state-of-the-art of the FPGA based PUF designs and their corresponding performances. We also elaborated the known attacks on FPGA based PUFs and corresponding countermeasures. Then four typical applications scenarios with practical examples of FPGA based implementations are discussed. Finally, since entropy estimation, security issues, CAD frameworks for design, test and evaluation of PUFs, environmental influences and hardware efficiency of PUFs have brought new challenges, it is suggested to develop effective solutions to address them.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] S. Singh, FPGA Market worth 11.0 billion by 2025 with a growing CAGR of 8.0%, 2019, https://www.marketsandmarkets.com/PressReleases/fpga.asp?gclid.
[2] I. Bolsens, G. Gielen, K. Roy, U. Schneider, All Programmable SOC FPGA for networking and computing in big data infrastructure, in: 2014 19th Asia and South Pacific Design Automation Conference (ASP-DAC), 2014, pp. 1–3.
[3] J. Zhang, G. Qu, Recent attacks and defenses on FPGA-based systems, ACM Trans. Reconfigurable Technol. Syst. 12 (3) (2019) 14:1–14:24.
[4] N.N. Anandakumar, S.K. Sanadhya, M.S. Hashmi, FPGA-Based true random number generation using programmable delays in oscillator-rings, IEEE Trans. Circuits Syst. II: Expr. Briefs (2019) 1–5.
[5] M. Bakiri, C. Guyeux, J.-F. cois Couchot, A.K. Oudjida, Survey on hardware implementation of random number generators on FPGA: Theory and experimental analyses, Comp. Sci. Rev. 27 (2018) 135–153.
[6] N.N. Anandakumar, M.P.L. Das, S.K. Sanadhya, M.S. Hashmi, Reconfigurable hardware architecture for authenticated key agreement protocol over binary edwards curve, ACM Trans. Reconfigurable Technol. Syst. 11 (2) (2018) 12:1–12:19.
[7] Xilinx, FPGA IFF Copy protection using dallas semiconductor/maxim DS2432 secure EEPROMs, 2010.
[8] P.S. Ravikanth, Physical One-Way Functions (PH.D. Thesis), Massachusetts Institute of Technology, Mar 2001.
[9] B. Gassend, D.E. Clarke, M. van Dijk, S. Devadas, Silicon physical random functions, in: Proceedings of the 9th ACM Conference on Computer and Communications Security, CCS 2002, ACM, 2002, pp. 148–160.
[10] R. Maes, A. Van Herrewege, I. Verbauwhede, PUFKY: A fully functional PUF-based cryptographic key generator, in: Cryptographic Hardware and Embedded Systems – CHES 2012: 14th International Workshop, Leuven, Belgium, September 9-12, 2012. Proceedings, Springer Berlin Heidelberg, CHES 2012, pp. 302–319.

[11] G.E. Suh, S. Devadas, Physical unclonable functions for device authentication and secret key generation, in: Proceedings of the 44th Design Automation Conference, DAC 2007, USA, June 4-8, 2007, IEEE, 2007, pp. 9–14.

[12] M.A. Usmani, S. Keshavarz, E. Matthews, L. Shannon, R. Tessier, D.E. Holcomb, Efficient PUF-based key generation in FPGAs using per-device configuration, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 27 (2) (2019) 364–375.

[13] S. Sutar, A. Raha, D. Kulkarni, R. Shorey, J. Tew, V. Raghunathan, D-PUF: An intrinsically reconfigurable DRAM PUF for device authentication and random number generation, ACM Trans. Embed. Comput. Syst. 17 (1) (2017) 17:1–17:31.

[14] S. Kalanadhabhatta, D. Kumar, K.K. Anumandla, S.A. Reddy, A. Acharyya, PUF-Based secure chaotic random number generator design methodology, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. (2020) 1–5.

[15] J. Guajardo, S.S. Kumar, G.-J. Schrijen, P. Tuyls, FPGA Intrinsic PUFs and their use for IP protection, in: Cryptographic Hardware and Embedded Systems - CHES 2007, 4727, Springer, 2007, pp. 63–80.

[16] D.B. Roy, S. Bhasin, I. Nikolić, D. Mukhopadhyay, Combining PUF with RLUTs: A two-party pay-per-device IP licensing scheme on FPGAs, ACM Trans. Embed. Comput. Syst. 18 (2) (2019) 12:1–12:22.

[17] D.E. Holcomb, W.P. Burleson, K. Fu, Power-up SRAM state as an identifying fingerprint and source of true random numbers, IEEE Trans. Comput. 58 (9) (2009) 1198–1210.

[18] A. Aysu, P. Schaumont, Hardware/software co-design of physical unclonable function based authentications on FPGAs, Microprocess. Microsyst. 39 (7) (2015) 589–597.

[19] U. Chatterjee, V. Govindan, R. Sadhukhan, D. Mukhopadhyay, R.S. Chakraborty, D. Mahata, M.M. Prabhu, Building PUF based authentication and key exchange protocol for IoT without explicit CRPs in verifier database, IEEE Trans. Dependable Secure Comput. 16 (3) (2019) 424–437.

[20] N.N. Anandakumar, Design, Implementation and Analysis of Efficient Hardware-based Security Primitives (Ph.d. dissertation), IIIT-Delhi, 2019.

[21] O. Günlü, T. Kernetzky, O. Iscan, V. Sidorenko, G. Kramer, R.F. Schaefer, Secure and reliable key agreement with physical unclonable functions, Entropy 20 (5) (2018) 340.

[22] K. Yang, D. Forte, M.M. Tehranipoor, CDTA: A comprehensive solution for counterfeit detection, traceability, and authentication in the IoT supply chain, ACM Trans. Des. Autom. Electron. Syst. 22 (3) (2017) 42:1–42:31.

[23] A. Babaei, G. Schiele, Physical unclonable functions in the internet of things: State of the art and open challenges, Sensors 19 (14) (2019) 3208.

[24] C. Marchand, L. Bossuet, U. Mureddu, N. Bochard, A. Cherkaoui, V. Fischer, Implementation and characterization of a physical unclonable function for IoT: A case study with the TERO-PUF, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 37 (1) (2018) 97–109.

[25] J. Zhang, X. Tan, Y. Zhang, W. Wang, Z. Qin, Frequency offset-based ring oscillator physical unclonable function, IEEE Trans. Multi-Scale Comput. Syst. 4 (2018) 711–721.

[26] W. Liu, L. Zhang, Z. Zhang, C. Gu, C. Wang, M. O'Neill, F. Lombardi, XOR-Based low-cost reconfigurable PUFs for IoT security, ACM Trans. Embed. Comput. Syst. 18 (3) (2019) 25:1–25:21.

[27] N. Menhorn, External secure storage using the PUF, 2018.

[28] K. Horovitz, R. Kenny, Intel FPGA secure device manager, 2018, https://apps.dtic.mil/dtic/tr/fulltext/u2/1052301.pdf.

[29] T. Speers, Polarfire non-volatile FPGA family delivers ground breaking value: Best-in-class security, 2018, https://www.microsemi.com/blog/2018/04/10/.

[30] Intrisic-ID Inc., https://www.intrinsic-id.com/. (Accessed Sep 2019).

[31] Lewis Innovative Technology Inc., http://ww5.lewisinnovative.com/. (Accessed Sep 2019).

[32] QuantumTrace LLC, http://www.quantumtrace.com/Technology/. (Accessed Sep 2019).

[33] Helion Technology Limited. https://www.heliontech.com/. (Accessed Sep 2019).

[34] R. Maes, I. Verbauwhede, Physically unclonable functions: A study on the state of the art and future research directions, in: Towards Hardware-Intrinsic Security: Foundations and Practice, Springer Berlin Heidelberg, Berlin, Heidelberg, 2010, pp. 3–37.

[35] N. Noor, H. Silva, Phase change memory for physical unclonable functions, in: M. Suri (Ed.), Applications of Emerging Memory Technology: Beyond Storage, Springer Singapore, 2020, pp. 59–91.

[36] A. Mutlu, K.J. Le, M. Celik, D. Tsien, G. Shyu, L. Yeh, An exploratory study on statistical timing analysis and parametric yield optimization, 2007, pp. 677–684.

[37] J.-L. Zhang, G. Qu, Y.-Q. Lv, Q. Zhou, A survey on silicon PUFs and recent advances in ring oscillator PUFs, J. Comput. Sci. Tech. 29 (4) (2014) 664–678.

[38] T. McGrath, I.E. Bagci, Z.M. Wang, U. Roedig, R.J. Young, A PUF taxonomy, Appl. Phys. Rev. 6 (1) (2019) 011303.

[39] D. Yamamoto, K. Sakiyama, M. Iwamoto, K. Ohta, M. Takenaka, K. Itoh, Variety enhancement of PUF responses using the locations of random outputting RS latches, J. Cryptogr. Eng. 3 (4) (2013) 197–211.

[40] R. Maes, P. Tuyls, I. Verbauwhede, Intrinsic PUFs from flip-flops on reconfigurable devices, in: 3rd Benelux Workshop Information and System Security, 2008, pp. 17.

[41] S.S. Kumar, J. Guajardo, R. Maes, G.J. Schrijen, P. Tuyls, Extended abstract: The butterfly PUF protecting IP on every FPGA, in: IEEE International Workshop on Hardware-Oriented Security and Trust, 2008, pp. 67–70.

[42] F. Tehranipoor, N. Karimian, W. Yan, J.A. Chandy, DRAM-Based intrinsic physically unclonable functions for system-level security and authentication, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 25 (3) (2017) 1085–1097.

[43] J. Kong, F. Koushanfar, Processor-based strong physical unclonable functions with aging-based response tuning, IEEE Trans. Emerg. Top. Comput. 2 (1) (2014) 16–29.

[44] K.S. Kumar, S. Sahoo, A. Mahapatra, A.K. Swain, K. Mahapatra, Microprocessor based physical unclonable function, in: 2017 IEEE International Symposium on Nanoelectronic and Information Systems (INIS), 2017, pp. 246–251.

[45] Y. Zheng, F. Zhang, S. Bhunia, DScanPUF: A delay-based physical unclonable function built into scan chain, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 24 (3) (2016) 1059–1070.

[46] J. Aarestad, P. Ortiz, D. Acharyya, J. Plusquellic, HELP: A hardware-embedded delay PUF, IEEE Design Test 30 (2) (2013) 17–25.

[47] D. Suzuki, K. Shimizu, The Glitch PUF: A new delay-PUF architecture exploiting glitch shapes, in: S. Mangard, F.-X. Standaert (Eds.), Cryptographic Hardware and Embedded Systems, CHES 2010, Springer Berlin Heidelberg, 2010, pp. 366–382.

[48] U. Rührmair, D.E. Holcomb, PUFs at a glance, in: Proceedings of the Conference on Design, Automation & Test in Europe, DATE '14, 2014, pp. 347:1–347:6.

[49] C. Gu, N. Hanley, M. O'neill, Improved reliability of FPGA-based PUF identification generator design, ACM Trans. Reconfigurable Technol. Syst. 10 (3) (2017) 20:1–20:23.

[50] M.M. Yu, M. Hiller, J. Delvaux, R. Sowell, S. Devadas, I. Verbauwhede, A lockdown technique to prevent machine learning on PUFs for lightweight authentication, IEEE Trans. Multi Scale Comput. Syst. 2 (3) (2016) 146–159.

[51] B. Gassend, M.V. Dijk, D. Clarke, E. Torlak, S. Devadas, P. Tuyls, Controlled physical random functions and applications, ACM Trans. Inf. Syst. Secur. 10 (4) (2008) 3:1–3:22.

[52] A. Maiti, V. Gunreddy, P. Schaumont, A systematic method to evaluate and compare the performance of physical unclonable functions, in: Embedded Systems Design with FPGAs, Springer New York, New York, NY, 2013, pp. 245–267.

[53] M.M. Yu, R. Sowell, A. Singh, D. M'Raïhi, S. Devadas, Performance metrics and empirical results of a PUF cryptographic key generation ASIC, in: 2012 IEEE International Symposium on Hardware-Oriented Security and Trust, 2012, pp. 108–115.

[54] W. Che, M. Martinez-Ramon, F. Saqib, J. Plusquellic, Delay model and machine learning exploration of a hardware-embedded delay PUF, in: IEEE Inter. Symposium on Hardware Oriented Security and Trust (HOST), 2018, pp. 153–158.

[55] C. Herder, M.D. Yu, F. Koushanfar, S. Devadas, Physical unclonable functions and applications: A tutorial, Proc. IEEE 102 (8) (2014) 1126–1141.

[56] M. van Dijk, U. Rührmair, Protocol attacks on advanced PUF protocols and countermeasures, in: 2014 Design, Automation Test in Europe Conference Exhibition (DATE), 2014, pp. 1–6.

[57] M. Potkonjak, V. Goudar, Public physical unclonable functions, Proc. IEEE 102 (8) (2014) 1142–1156.

[58] W. Liang, B. Liao, J. Long, Y. Jiang, L. Peng, Study on PUF based secure protection for IC design, Microprocess. Microsyst. 45 (2016) 56–66.

[59] N.A. Anagnostopoulos, S. Katzenbeisser, J. Chandy, F. Tehranipoor, An overview of DRAM-based security primitives, Cryptography 2 (2) (2018).

[60] C.-H. Chang, Y. Zheng, L. Zhang, A retrospective and a look forward: Fifteen years of physical unclonable function advancement, IEEE Circuits Syst. Mag. 17 (3) (2017) 32–62.

[61] H. Ning, F. Farha, A. Ullah, L. Mao, Physical unclonable function: architectures, applications and challenges for dependable security, IET Circuits, Devices Syst. 14 (4) (2020) 407–424.

[62] J. Delvaux, R. Peeters, D. Gu, I. Verbauwhede, A survey on lightweight entity authentication with strong PUFs, ACM Comput. Surv. 48 (2) (2015) 26:1–26:42.

[63] J. Rajendran, R. Karri, J.B. Wendt, M. Potkonjak, N. McDonald, G.S. Rose, B. Wysocki, Nano meets security: Exploring nanoelectronic devices for security applications, Proc. IEEE 103 (5) (2015) 829–849.

[64] Y. Gao, D.C. Ranasinghe, S.F. Al-Sarawi, O. Kavehei, D. Abbott, Emerging physical unclonable functions with nanotechnology, IEEE Access 4 (2016) 61–80.

[65] Y. Hori, T. Yoshida, T. Katashita, A. Satoh, Quantitative and statistical performance evaluation of arbiter physical unclonable functions on FPGAs, in: International Conference on Reconfigurable Computing and FPGAs, 2010, pp. 298–303.

[66] N.Q.M. Noor, S.M. Daud, N.A. Ahmad, N. Maarop, Defense mechanisms against machine learning modeling attacks on strong physical unclonable functions for IOT authentication: A review, Int. J. Adv. Comput. Sci. Appl. 8 (10) (2017).

[67] J. Delvaux, D. Gu, D. Schellekens, I. Verbauwhede, Helper data algorithms for PUF-based key generation: Overview and analysis, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 34 (6) (2015) 889–902.

[68] C. Yehoshuva, R. Raja Adhithan, N. Nalla Anandakumar, A survey of security attacks on silicon based weak PUF architectures, in: Security in Computing and Communications–6th International Symposium SSCC 2020, Springer, 2020, pp. 483–494.

[69] S. Chowdhury, A. Covic, R.Y. Acharya, S. Dupee, F. Ganji, D. Forte, Physical security in the post-quantum era: A survey on side-channel analysis, random number generators, and physically unclonable functions, 2020, arXiv:2005.04344.

[70] J.W. Lee, D. Lim, B. Gassend, G.E. Suh, M. van Dijk, S. Devadas, A technique to build a secret key in integrated circuits for identification and authentication applications, in: IEEE Symposium on VLSI Circuits. Digest of Technical Papers, 2004, pp. 176–179.

[71] L. Bossuet, X.T. Ngo, Z. Cherif, V. Fischer, A PUF based on a transient effect ring oscillator and insensitive to locking phenomenon, IEEE Trans. Emerg. Top. Comput. 2 (1) (2014) 30–36.

[72] J.H. Anderson, A PUF design for secure FPGA-based embedded systems, in: 2010 15th Asia and South Pacific Design Automation Conference (ASP-DAC), 2010, pp. 1–6.

[73] Y. Hori, H. Kang, T. Katashita, A. Satoh, Pseudo-LFSR PUF: A compact, efficient and reliable physical unclonable function, in: 2011 International Conference on Reconfigurable Computing and FPGAs, 2011, pp. 223–228.

[74] S. Katzenbeisser, Ü. Koçabas, V. van der Leest, A. Sadeghi, G. Schrijen, H. Schröder, C. Wachsmann, Recyclable PUFs: Logically reconfigurable PUFs, in: Cryptographic Hardware and Embedded Systems - CHES 2011, 1, Springer, 2011, pp. 374–389.

[75] M. Majzoobi, F. Koushanfar, S. Devadas, FPGA PUF using programmable delay lines, in: IEEE International Workshop on Information Forensics and Security, WIFS, IEEE, 2010, pp. 1–6.

[76] C. Gu, W. Liu, Y. Cui, N. Hanley, M. O'Neill, F. Lombardi, A flip-flop based arbiter physical unclonable function (APUF) design with high entropy and uniqueness for FPGA implementation, IEEE Trans. Emer. Top. Comput. (TETC) (2019).

[77] S. Morozov, A. Maiti, P. Schaumont, An analysis of delay based PUF implementations on FPGA, in: Reconfigurable Computing: Architectures, Tools and Applications, Springer Berlin Heidelberg, Berlin, Heidelberg, 2010, pp. 382–387.

[78] D.P. Sahoo, R.S. Chakraborty, D. Mukhopadhyay, Towards ideal arbiter PUF design on xilinx FPGA: A practitioner's perspective, in: Euromicro Conference on Digital System Design, 2015, pp. 559–562.

[79] N.N. Anandakumar, M.S. Hashmi, S.K. Sanadhya, Compact implementations of FPGA-based PUFs with enhanced performance, in: 30th International Conference on VLSI Design and 16th International Conference on Embedded Systems (VLSID), 2017, pp. 161–166.

[80] D.P. Sahoo, D. Mukhopadhyay, R.S. Chakraborty, P.H. Nguyen, A multiplexer-based arbiter PUF composition with enhanced reliability and security, IEEE Trans. Comput. 67 (3) (2018) 403–417.

[81] S.V.S. Avvaru, Z. Zeng, K.K. Parhi, Homogeneous and heterogeneous feed-forward XOR physical unclonable functions, IEEE Trans. Inf. Forensics Secur. 15 (2020) 2485–2498.

[82] M. Majzoobi, F. Koushanfar, M. Potkonjak, Techniques for design and implementation of secure reconfigurable PUFs, ACM Trans. Reconfigurable Technol. Syst. (TRETS) 2 (1) (2009) 5.

[83] P. Nguyen, D. Sahoo, C. Jin, K. Mahmood, U. Rührmair, M. van Dijk, The interpose PUF: Secure PUF design against state-of-the-art machine learning attacks, IACR Trans. Cryptogr. Hardw. Embed. Syst. 2019 (4) (2019) 243–290.

[84] P. Santikellur, A. Bhattacharyay, R.S. Chakraborty, Deep learning based model building attacks on arbiter puf compositions, 2019, Cryptology ePrint Archive, Report 2019/566. https://eprint.iacr.org/2019/566.

[85] B. Habib, K. Gaj, J.P. Kaps, FPGA PUF based on programmable LUT delays, in: Euromicro Conference on Digital System Design, 2013, pp. 697–704.

[86] A. Maiti, P. Schaumont, Improved ring oscillator PUF: an FPGA-friendly secure primitive, J. Cryptology 24 (2011) 375–397.

[87] X. Xin, J. Kaps, K. Gaj, A configurable ring-oscillator-based PUF for Xilinx FPGAs, in: 2011 14th Euromicro Conference on Digital System Design, 2011, pp. 651–657.

[88] Z. Cherif, J. Danger, S. Guilley, L. Bossuet, An easy-to-design PUF based on a single oscillator: The Loop PUF, in: 2012 15th Euromicro Conference on Digital System Design, 2012, pp. 156–162.

[89] H. Yu, P.H.W. Leong, Q. Xu, An FPGA chip identification generator using configurable ring oscillators, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 20 (12) (2012) 2198–2207.

[90] Y. Cui, C. Wang, W. Liu, Y. Yu, M. O'Neill, F. Lombardi, Low-cost configurable ring oscillator PUF with improved uniqueness, in: 2016 IEEE International Symposium on Circuits and Systems (ISCAS), 2016, pp. 558–561.

[91] A.S. Chauhan, V. Sahula, A.S. Mandal, Novel randomized biased placement for FPGA based robust random number generator with enhanced uniqueness, in: 32nd International Conference on VLSI Design (VLSID), 2019, pp. 353–358.

[92] D. Merli, F. Stumpf, C. Eckert, Improving the quality of ring oscillator PUFs on FPGAs, in: WESS 2010, 2010.

[93] W. Yan, C. Jin, F. Tehranipoor, J.A. Chandy, Phase calibrated ring oscillator PUF design and implementation on FPGAs, in: 27th International Conference on Field Programmable Logic and Applications (FPL), 2017, pp. 1–8.

[94] Mingze Gao, Khai Lai, Gang Qu, A highly flexible ring oscillator PUF, in: 2014 51st ACM/EDAC/IEEE Design Automation Conference (DAC), 2014, pp. 1–6.

[95] M. Choudhury, N. Pundir, M. Niamat, M. Mustapa, Analysis of a novel stage configurable ROPUF design, in: 2017 IEEE 60th International Midwest Symposium on Circuits and Systems (MWSCAS), 2017, pp. 942–945.

[96] T. Tanamoto, S. Yasuda, S. Takaya, S. Fujita, Physically unclonable function using an initial waveform of ring oscillators, IEEE Trans. Circuits Syst. II: Express Briefs 64 (7) (2017) 827–831.

[97] S.R. Sahoo, K.S. Kumar, K. Mahapatra, A novel current controlled configurable RO PUF with improved security metrics, Integr. 58 (2017) 401–410.

[98] Z. Pang, J. Zhang, Q. Zhou, S. Gong, X. Qian, B. Tang, Crossover ring oscillator PUF, in: 2017 18th International Symposium on Quality Electronic Design (ISQED), 2017, pp. 237–243.

[99] B. Srinivasu, P. Vikramkumar, A. Chattopadhyay, K. Lam, CoLPUF : A Novel Configurable LFSR-based PUF, in: 2018 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS), 2018, pp. 358–361.

[100] Y. Cui, C. Gu, C. Wang, M. O'Neill, W. Liu, Ultra-lightweight and reconfigurable tristate inverter based physical unclonable function design, IEEE Access 6 (2018) 28478–28487.

[101] Y. Cui, Y. Chen, C. Wang, C. Gu, M. O'Neill, W. Liu, Programmable ring oscillator PUF based on switch matrix, in: 2020 IEEE International Symposium on Circuits and Systems (ISCAS), 2020, pp. –4.

[102] Q. Chen, G. Csaba, P. Lugli, U. Schlichtmann, U. Rührmair, The Bistable Ring PUF: A new architecture for strong physical unclonable functions, in: 2011 IEEE International Symposium on Hardware-Oriented Security and Trust, 2011, pp. 134–141.

[103] Y. Nozaki, Y. Ikezaki, M. Yoshikawa, Evaluation of PLFSR PUF with several implementation methods for FPGA, in: 2017 IEEE International Meeting for Future of Electron Devices, Kansai (IMFEDK), 2017, pp. 46–47.

[104] R. Maes, P. Tuyls, I. Verbauwhede, Intrinsic PUFs from flip-flops on reconfigurable devices, in: 3rd Benelux Workshop on Information and System Security (WISSec 2008), Vol. 17, 2008, p. 2008.

[105] D.E. Holcomb, W.P. Burleson, K. Fu, Initial SRAM state as a fingerprint and source of true random numbers for RFID tags, in: Proceedings of the Conference on RFID Security, 2007.

[106] T. Güneysu, Using data contention in dual-ported memories for security applications, J. Signal Process. Syst. 67 (1) (2012) 15–29.

[107] Y. Su, J. Holleman, B. Otis, A 1.6pJ/bit 96% Stable Chip-ID Generating Circuit using Process Variations, in: 2007 IEEE International Solid-State Circuits Conference. Digest of Technical Papers, 2007, pp. 406–611.

[108] A. Ardakani, S.B. Shokouhi, A. Reyhani-Masoleh, Improving performance of FPGA-based SR-latch PUF using transient effect ring oscillator and programmable delay lines, Integration 62 (2018) 371–381.

[109] B. Habib, J. Kaps, K. Gaj, Efficient SR-Latch PUF, in: Proceedings of the Applied Reconfigurable Computing - 11th International Symposium, ARC 2015, Bochum, Germany, April 13-17, 2015, 9040, Springer, 2015, pp. 205–216.

[110] A. Stanciu, M.N. Cirstea, F.D. Moldoveanu, Analysis and evaluation of PUF-based SoC designs for security applications, IEEE Trans. Ind. Electron. 63 (9) (2016) 5699–5708.

[111] S. Khoshroo, Design and evaluation of FPGA-based hybrid physically unclonable functions, in: Electronic Thesis and Dissertation Repository, 2013, http://ir.lib.uwo.ca/etd/1281.

[112] N.N. Anandakumar, M.S. Hashmi, S.K. Sanadhya, Efficient and lightweight FPGA-based hybrid PUFs with improved performance, Microprocess. Microsyst. 77 (2020) 103180.

[113] D.P. Sahoo, D. Mukhopadhyay, R.S. Chakraborty, Design of low area-overhead ring oscillator PUF with large challenge space, in: IEEE International Conference on Reconfigurable Computing and FPGAs (ReConFig), 2013.

[114] D.P. Sahoo, S. Saha, D. Mukhopadhyay, R.S. Chakraborty, H. Kapoor, Composite PUF: A new design paradigm for physically unclonable functions on FPGA, in: IEEE International Symposium on Hardware-Oriented Security and Trust (HOST), 2014.

[115] T. Machida, D. Yamamoto, M. Iwamoto, K. Sakiyama, A new arbiter PUF for enhancing unpredictability on FPGA, Sci. World J. 2015 (13) (2015).

[116] U. Rührmair, F. Sehnke, J. Sölter, G. Dror, S. Devadas, J. Schmidhuber, Modeling attacks on physical unclonable functions, in: Proceedings of the 17th ACM Conference on Computer and Communications Security, in: CCS '10, ACM, 2010, pp. 237–249.

[117] U. Rührmair, M. van Dijk, PUFs in security protocols: Attack models and security evaluations, in: 2013 IEEE Symposium on Security and Privacy, 2013, pp. 286–300.

[118] U. Rührmair, J. Sölter, F. Sehnke, X. Xu, A. Mahmoud, V. Stoyanova, G. Dror, J. Schmidhuber, W.P. Burleson, S. Devadas, PUF modeling attacks on simulated and silicon data, IEEE Trans. Inf. Forensics Secur. 8 (11) (2013) 1876–1891.

[119] R.R. Adhithan, N.N. Anandakumar, Modeling attacks and efficient countermeasures on interpose PUF, in: Foundations and Practice of Security - 13th International Symposium, FPS 2020, Montreal, QC, Canada, December 1-3, 2020, Revised Selected Papers, in: Lecture Notes in Computer Science, 12637, Springer, 2020, pp. 149–162.

[120] J. Miskelly, C. Gu, Q. Ma, Y. Cui, W. Liu, M. O'Neill, Modelling attack analysis of configurable ring oscillator (CRO) PUF Designs, in: 2018 IEEE 23rd International Conference on Digital Signal Processing (DSP), 2018, pp. 1–5.

[121] J. Delvaux, Machine-learning attacks on PolyPUFs, OB-PUFs, RPUFs, LHS-PUFs, and PUF-FSMs, IEEE Trans. Inf. Forensics Secur. 14 (8) (2019) 2043–2058.

[122] N. Wisiol, C. Mühl, N. Pirnay, P.H. Nguyen, M. Margraf, J. Seifert, M. van Dijk, U. Rührmair, Splitting the interpose PUF: a novel modeling attack strategy, IACR Trans. Cryptogr. Hardw. Embed. Syst. 2020 (3) (2020) 97–120.

[123] J. Ye, Q. Guo, Y. Hu, H. Li, X. Li, Modeling attacks on strong physical unclonable functions strengthened by random number and weak PUF, in: IEEE 36th VLSI Test Symposium (VTS), 2018, pp. 1–6.

[124] X. Xu, W. Burleson, Hybrid side-channel/machine-learning attacks on PUFs: A new threat?, in: 2014 Design, Automation Test in Europe Conference Exhibition (DATE), 2014, pp. 1–6.

[125] D.P. Sahoo, P.H. Nguyen, D. Mukhopadhyay, R.S. Chakraborty, A case of lightweight PUF constructions: Cryptanalysis and machine learning attacks, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 34 (8) (2015) 1334–1343.

[126] G.T. Becker, The gap between promise and reality: On the insecurity of XOR arbiter PUFs, in: T. Güneysu, H. Handschuh (Eds.), Cryptographic Hardware and Embedded Systems – CHES 2015, Springer Berlin Heidelberg, 2015, pp. 535–555.

[127] G.T. Becker, On the pitfalls of using arbiter-PUFs as building blocks, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 34 (8) (2015) 1295–1307, http://dx.doi.org/10.1109/TCAD.2015.2427259.

[128] M. Khalafalla, C. Gebotys, PUFs deep attacks: Enhanced modeling attacks using deep learning techniques to break the security of double arbiter PUFs, in: 2019 Design, Automation Test in Europe Conference Exhibition (DATE), 2019, pp. 204–209.

[129] H. Awano, T. Iizuka, M. Ikeda, PUFNet: A deep neural network based modeling attack for physically unclonable function, in: 2019 IEEE International Symposium on Circuits and Systems (ISCAS), 2019, pp. 1–4.

[130] R. Yashiro, T. Machida, M. Iwamoto, K. Sakiyama, Deep-learning-based security evaluation on authentication systems using arbiter PUF and its variants, in: Advances in Information and Computer Security - 11th International Workshop on Security, IWSEC 2016, in: Lecture Notes in Computer Science, 9836, Springer, 2016, pp. 267–285.

[131] J. Shi, Y. Lu, J. Zhang, Approximation attacks on strong PUFs, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 39 (10) (2020) 2138–2151.

[132] J. Tobisch, A. Aghaie, G.T. Becker, Combining optimization objectives: New modeling attacks on strong PUFs, IACR Trans. Cryptogr. Hardw. Embed. Syst. 2021 (2) (2021) 357–389.

[133] F. Ganji, On the learnability of physically unclonable functions (Ph.D. thesis), Technical University of Berlin, Germany, 2017, https://nbn-resolving.org/urn:nbn:de:101:1-201804179933.

[134] F. Ganji, D. Forte, J. Seifert, Pufmeter a property testing tool for assessing the robustness of physically unclonable functions to machine learning attacks, IEEE Access (2019) 122513–122521.

[135] D. Chatterjee, D. Mukhopadhyay, A. Hazra, PUF-G: A CAD Framework for Automated Assessment of Provable Learnability from Formal PUF Representations, in: 2020 IEEE/ACM International Conference on Computer Aided Design (ICCAD), 2020, pp. 1–9.

[136] J. Zhang, C. Shen, Set-based obfuscation for strong PUFs against machine learning attacks, IEEE Trans. Circuits Syst. I. Regul. Pap. 68 (1) (2021) 288–300.

[137] H. Awano, T. Sato, Ising-PUF: A machine learning attack resistant PUF featuring lattice like arrangement of Arbiter-PUFs, in: 2018 Design, Automation Test in Europe Conference Exhibition (DATE), 2018, pp. 1447–1452.

[138] C. Gu, C.-H. Chang, W. Liu, S. Yu, Y. Wang, M. O'Neill, A modeling attack resistant deception technique for securing lightweight-PUF-based authentication, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 40 (6) (2021) 1183–1196.

[139] Q. Ma, C. Gu, N. Hanley, C. Wang, W. Liu, M. O'Neill, A machine learning attack resistant multi-PUF design on FPGA, in: Proceedings of the 23rd Asia and South Pacific Design Automation Conference, in: ASPDAC '18, IEEE Press, 2018, pp. 97–104.

[140] M.S. Mispan, H. Su, M. Zwolinski, B. Halak, Cost-efficient design for modeling attacks resistant PUFs, in: 2018 Design, Automation Test in Europe Conference Exhibition (DATE), 2018, pp. 467–472.

[141] Z. Wu, H. Patel, M. Sachdev, M.V. Tripunitara, Strengthening PUFs using Composition, in: 2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2019, pp. 1–8.

[142] Q. Wang, M. Gao, G. Qu, A machine learning attack resistant dual-mode PUF, in: Proceedings of the 2018 on Great Lakes Symposium on VLSI, in: GLSVLSI '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 177–182.

[143] S.-J. Wang, Y.-S. Chen, K.S.-M. Li, Modeling attack resistant PUFs based on adversarial attack against machine learning, IEEE J. Emerg. Sel. Top. Circuits Syst. (2021) 1.

[144] E. Dubrova, O. Näslund, B. Degen, A. Gawell, Y. Yu, CRC-PUF: A machine learning attack resistant lightweight PUF construction, in: 2019 IEEE European Symposium on Security and Privacy Workshops (EuroS PW), 2019, pp. 264–271.

[145] H. Gu, M. Potkonjak, Securing interconnected PUF network with reconfigurability, in: 2018 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), 2018, pp. 231–234.

[146] M. Majzoobi, M. Rostami, F. Koushanfar, D.S. Wallach, S. Devadas, Slender PUF Protocol: A lightweight, robust, and secure authentication by substring matching, in: 2012 IEEE Symposium on Security and Privacy Workshops, 2012, pp. 33–44.

[147] J. Ye, Y. Hu, X. Li, RPUF: Physical unclonable function with randomized challenge to resist modeling attack, in: 2016 IEEE Asian Hardware-Oriented Security and Trust (AsianHOST), 2016, pp. 1–6.

[148] M. Rostami, M. Majzoobi, F. Koushanfar, D.S. Wallach, S. Devadas, Robust and reverse-engineering resilient PUF authentication and key-exchange by substring matching, IEEE Trans. Emerg. Top. Comput. 2 (1) (2014) 37–49.

[149] Y. Gao, G. Li, H. Ma, S.F. Al-Sarawi, O. Kavehei, D. Abbott, D.C. Ranasinghe, Obfuscated challenge-response: A secure lightweight authentication mechanism for PUF-based pervasive devices, in: 2016 IEEE International Conference on Pervasive Computing and Communication Workshops (PerCom Workshops), 2016, pp. 1–6.

[150] M.-D. Yu, D. M'Raï hi, I. Verbauwhede, S. Devadas, A noise bifurcation architecture for linear additive physical functions, in: 2014 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST), 2014, pp. 124–129.

[151] J. Ye, Y. Hu, X. Li, OPUF: Obfuscation logic based physical unclonable function, in: 2015 IEEE 21st International on-Line Testing Symposium (IOLTS), 2015, pp. 156–161.

[152] A. Wang, W. Tan, Y. Wen, Y. Lao, NoPUF: A novel PUF design framework toward modeling attack resistant PUFs, IEEE Trans. Circuits Syst. I. Regul. Pap. 68 (6) (2021) 2508–2521.

[153] A. Vijayakumar, V.C. Patil, C.B. Prado, S. Kundu, Machine learning resistant strong PUF: Possible or a pipe dream? in: 2016 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), 2016, pp. 19–24.

[154] N. Shah, D. Chatterjee, B. Sapui, D. Mukhopadhyay, A. Basu, Introducing recurrence in strong PUFs for enhanced machine learning attack resistance, IEEE J. Emerg. Sel. Top. Circuits Syst. (2021) 1.

[155] S. Tajik, E. Dietz, S. Frohmann, H. Dittrich, D. Nedospasov, C. Helfmeier, J.-P. Seifert, C. Boit, H.-W. Hübers, Photonic side-channel analysis of arbiter PUFs, J. Cryptol. 30 (2) (2017) 550–571.

[156] D. Nedospasov, J. Seifert, C. Helfmeier, C. Boit, Invasive PUF Analysis, in: 2013 Workshop on Fault Diagnosis and Tolerance in Cryptography, 2013, pp. 30–38.

[157] S. Tajik, H. Lohrke, F. Ganji, J. Seifert, C. Boit, Laser fault attack on physically unclonable functions, in: 2015 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC), 2015, pp. 85–96.

[158] D.P. Sahoo, S. Patranabis, D. Mukhopadhyay, R.S. Chakraborty, Fault tolerant implementations of delay-based physically unclonable functions on FPGA, in: 2016 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC), 2016, pp. 87–101.

[159] D. Karakoyunlu, B. Sunar, Differential template attacks on PUF enabled cryptographic devices, in: IEEE International Workshop on Information Forensics and Security, 2010, pp. 1–6.

[160] D. Merli, J. Heyszl, B. Heinz, D. Schuster, F. Stumpf, G. Sigl, Localized electromagnetic analysis of RO PUFs, in: IEEE International Symposium on Hardware-Oriented Security and Trust (HOST), 2013, pp. 19–24.

[161] A. Mahmoud, U. Rührmair, M. Majzoobi, F. Koushanfar, Combined modeling and side channel attacks on strong PUFs, IACR Cryptology EPrint Archive 2013 (2013) 632.

[162] G.T. Becker, R. Kumar, Active and passive side-channel attacks on delay based PUF designs, IACR Cryptol. ePrint Arch. 2014 (2014) 287, https://eprint.iacr.org/2014/287.

[163] L. Tebelmann, J.-L. Danger, M. Pehl, Self-secured PUF: Protecting the loop PUF by masking, 2020, Cryptology ePrint Archive, Report 2020/145, https://eprint.iacr.org/2020/145.

[164] J. Delvaux, I. Verbauwhede, Side channel modeling attacks on 65nm arbiter PUFs exploiting CMOS device noise, in: 2013 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST), 2013, pp. 137–142.

[165] U. Rührmair, X. Xu, J. Sölter, A. Mahmoud, M. Majzoobi, F. Koushanfar, W. Burleson, Efficient power and timing side channels for physical unclonable functions, in: L. Batina, M. Robshaw (Eds.), Cryptographic Hardware and Embedded Systems – CHES 2014, Springer Berlin Heidelberg, 2014, pp. 476–492.

[166] L. Tebelmann, M. Pehl, V. Immler, Side-channel analysis of the TERO PUF, in: International Workshop on Constructive Side-Channel Analysis and Secure Design, Springer, 2019, pp. 43–60.

[167] A. Aghaie, A. Moradi, TI-PUF: Toward side-channel resistant physical unclonable functions, IEEE Trans. Inf. Forensics Secur. 15 (2020) 3470–3481.

[168] C. Helfmeier, C. Boit, D. Nedospasov, J. Seifert, Cloning Physically Unclonable Functions, in: 2013 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST), 2013, pp. 1–6.

[169] S. Zeitouni, Y. Oren, C. Wachsmann, P. Koeberl, A.-R. Sadeghi, Remanence decay side-channel: The PUF case, IEEE Trans. Inf. Forensics Secur. 11 (6) (2016) 1106–1116.

[170] M. Gao, K. Lai, J. Zhang, G. Qu, A. Cui, Q. Zhou, Reliable and anti-cloning PUFs based on configurable ring oscillators, in: 2015 14th International Conference on Computer-Aided Design and Computer Graphics (CAD/Graphics), 2015, pp. 194–201.
[171] X. Xu, U. Rührmair, D.E. Holcomb, W. Burleson, Security evaluation and enhancement of bistable ring PUFs, in: Radio Frequency Identification, Springer, 2015, pp. 3–16.
[172] A. Wild, G.T. Becker, T. Güneysu, A fair and comprehensive large-scale analysis of oscillation-based PUFs for FPGAs, in: 27th International Conference on Field Programmable Logic and Applications (FPL), 2017, pp. 1–7.
[173] R. Hesselbarth, F. Wilde, C. Gu, N. Hanley, Large scale RO PUF analysis over slice type, evaluation time and temperature on 28nm Xilinx FPGAs, in: IEEE Inter. Symp. on Hardware Oriented Security and Trust (HOST), 2018, pp. 126–133.
[174] T. Ignatenko, G. j. Schrijen, B. Skoric, P. Tuyls, F. Willems, Estimating the secrecy-rate of physical unclonable functions with the context-tree weighting method, in: IEEE International Symposium on Information Theory, 2006, pp. 499–503.
[175] V. van der Leest, G.-J. Schrijen, H. Handschuh, P. Tuyls, Hardware intrinsic security from d flip-flops, in: Proceedings of the Fifth ACM Workshop on Scalable Trusted Computing, ACM, 2010, pp. 53–62.
[176] M.S. Turan, E. Barker, J. Kelsey, K.A. McKay, M.L. Baish, M. Boyle, Recommendation for the entropy sources used for random bit generation, NIST Special Publ. 800 (90B) (2018).
[177] C. Gu, C.H. Chang, W. Liu, N. Hanley, J. Miskelly, M. O'Neill, A large scale comprehensive evaluation of single-slice ring oscillator and picopuf bit cells on 28nm xilinx FPGAs, in: Proceedings of the 3rd ACM Workshop on Attacks and Solutions in Hardware Security Workshop, in: ASHES'19, 2019, pp. 101–106.
[178] K. Chuang, E. Bury, R. Degraeve, B. Kaczer, D. Linten, I. Verbauwhede, A physically unclonable function using soft oxide breakdown featuring 0% native BER and 51.8 fJ/bit in 40-nm CMOS, IEEE J. Solid-State Circuits 54 (10) (2019) 2765–2776.
[179] A. Mills, S. Vyas, M. Patterson, C. Sabotta, P. Jones, J. Zambreno, Design and evaluation of a delay-based FPGA physically unclonable function, in: 2012 IEEE 30th International Conference on Computer Design (ICCD), 2012, pp. 143–146.
[180] C. Gu, W. Liu, N. Hanley, R. Hesselbarth, M. O'Neill, A theoretical model to link uniqueness and min-entropy for PUF evaluations, IEEE Trans. Comput. 68 (2) (2019) 287–293.
[181] A. Maiti, P. Schaumont, The impact of aging on a physical unclonable function, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 22 (9) (2014) 1854–1864.
[182] H. Dogan, D. Forte, M.M. Tehranipoor, Aging analysis for recycled FPGA detection, in: 2014 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT), 2014, pp. 171–176.
[183] M.T. Rahman, F. Rahman, D. Forte, M. Tehranipoor, An aging-resistant RO-PUF for reliable key generation, IEEE Trans. Emerg. Top. Comput. 4 (3) (2016) 335–348.
[184] H. Kang, Y. Hori, T. Katashita, M. Hagiwara, K. Iwamura, Cryptographie key generation from PUF data using efficient fuzzy extractors, in: 16th International Conference on Advanced Communication Technology, 2014, pp. 23–26.
[185] B. Jarvis, K. Gaj, Selection of an error-correcting code for FPGA-based physical unclonable functions, in: 2017 International Conference on Field Programmable Technology (ICFPT), 2017, pp. 243–246.
[186] M.D. Yu, S. Devadas, Secure and robust error correction for physical unclonable functions, IEEE Design Test Comput. 27 (1) (2010) 48–65.
[187] V. van der Leest, B. Preneel, E. van der Sluis, Soft decision error correction for compact memory-based PUFs using a single enrollment, in: Cryptographic Hardware and Embedded Systems – CHES 2012, Springer Berlin Heidelberg, 2012, pp. 268–282.
[188] C. Bösch, J. Guajardo, A.-R. Sadeghi, J. Shokrollahi, P. Tuyls, Efficient helper data key extractor on FPGAs, in: Cryptographic Hardware and Embedded Systems – CHES 2008, Springer Berlin Heidelberg, 2008, pp. 181–197.
[189] M. Hiller, L.R. Lima, G. Sigl, Seesaw: An Area-Optimized FPGA Viterbi Decoder for PUFs, in: 2014 17th Euromicro Conference on Digital System Design, 2014, pp. 387–393.
[190] J. Zhang, G. Qu, Physical unclonable function-based key sharing via machine learning for IoT security, IEEE Trans. Ind. Electron. 67 (8) (2020) 7025–7033.
[191] T. Kean, Cryptographic rights management of FPGA intellectual property cores, in: Proceedings of the 2002 ACM/SIGDA Tenth International Symposium on Field-Programmable Gate Arrays, Association for Computing Machinery, New York, NY, USA, 2002, pp. 113–118.
[192] E. Simpson, P. Schaumont, Offline hardware/software authentication for reconfigurable platforms, in: Cryptographic Hardware and Embedded Systems - CHES 2006, Springer Berlin Heidelberg, 2006, pp. 311–323.
[193] J. Guajardo, S.S. Kumar, G. Schrijen, P. Tuyls, Brand and IP protection with physical unclonable functions, in: 2008 IEEE International Symposium on Circuits and Systems, 2008, pp. 3186–3189.
[194] J. Guajardo, S.S. Kumar, G. Schrijen, P. Tuyls, Physical unclonable functions and public-key crypto for FPGA IP protection, in: 2007 International Conference on Field Programmable Logic and Applications, 2007, pp. 189–195.
[195] M.E.S. Elrabaa, M.A. Al-Asli, M.H. Abu-Amara, A protection and pay-per-use licensing scheme for on-cloud FPGA circuit IPs, ACM Trans. Reconfigurable Technol. Syst. 12 (3) (2019).
[196] J. Kokila, N. Ramasubramanian, Enhanced authentication using hybrid PUF with FSM for protecting IPs of SoC FPGAs, J. Electronic Testing 35 (4) (2019) 543–558.
[197] J.X. Zheng, M. Potkonjak, A digital PUF-based IP protection architecture for network embedded systems, in: 2014 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS), 2014, pp. 255–256.
[198] J. Zhang, Y. Lin, Y. Lyu, G. Qu, A PUF-FSM binding scheme for FPGA IP protection and pay-per-device licensing, IEEE Trans. Inf. Forensics Secur. 10 (6) (2015) 1137–1150.
[199] L. Parrilla, E. Castillo, D.P. Morales, A. García, Hardware activation by means of PUFs and elliptic curve cryptography in field-programmable devices, Electronics 5 (1) (2016).
[200] J.B. Wendt, M. Potkonjak, Hardware obfuscation using PUF-based logic, in: 2014 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2014, pp. 270–271.
[201] J. Zhang, A practical logic obfuscation technique for hardware security, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 24 (3) (2016) 1193–1197.
[202] M.J. Atallah, E.D. Bryant, J.T. Korb, J.R. Rice, Binding software to specific native hardware in a VM environment: The puf challenge and opportunity, in: VMSec '08, ACM, New York, NY, USA, 2008, pp. 45–48.
[203] M.A. Gora, A. Maiti, P. Schaumont, A flexible design flow for software IP binding in FPGA, IEEE Trans. Ind. Inf. 6 (4) (2010) 719–728.
[204] F. Kohnhäuser, A. Schaller, S. Katzenbeisser, PUF-Based software protection for low-end embedded devices, in: M. Conti, M. Schunter, I. Askoxylakis (Eds.), Trust and Trustworthy Computing, Springer International Publishing, Cham, 2015, pp. 3–21.
[205] P. Qiu, Y. Lyu, D. Zhai, D. Wang, J. Zhang, X. Wang, G. Qu, Physical unclonable functions-based linear encryption against code reuse attacks, in: 2016 53nd ACM/EDAC/IEEE Design Automation Conference (DAC), 2016, pp. 1–6.
[206] J. Zhang, B. Qi, Z. Qin, G. Qu, HCIC: Hardware-assisted control-flow integrity checking, IEEE Internet Things J. 6 (1) (2019) 458–471.
[207] W. Xiong, A. Schaller, S. Katzenbeisser, J. Szefer, Software protection using dynamic PUFs, IEEE Trans. Inf. Forensics Secur. 15 (2020) 2053–2068.
[208] D. Li, Z. Lu, X. Zou, Z. Liu, PUFKEY: A high-security and high-throughput hardware true random number generator for sensor networks, Sensors 15 (10) (2015) 26251–26266.
[209] V. van der Leest, E. van der Sluis, G.-J. Schrijen, P. Tuyls, H. Handschuh, Efficient implementation of true random number generator based on SRAM PUFs, in: Cryptography and Security: From Theory To Applications, Springer Berlin Heidelberg, 2012, pp. 300–318.
[210] S. Chen, B. Li, C. Zhou, FPGA Implementation of SRAM PUFs based cryptographically secure pseudo-random number generator, Microprocess. Microsyst. 59 (2018) 57–68.
[211] M. Barbareschi, G. Di Natale, L. Torres, A. Mazzeo, A ring oscillator-based identification mechanism immune to aging and external working conditions, IEEE Trans. Circuits Syst. I. Regul. Pap. 65 (2) (2018) 700–711.
[212] J. Long, W. Liang, K. Li, D. Zhang, M. Tang, H. Luo, PUF-Based anonymous authentication scheme for hardware devices and IPs in edge computing environment, IEEE Access 7 (2019) 124785–124796.
[213] A. Aysu, E. Gulcan, D. Moriyama, P. Schaumont, M. Yung, End-to-end design of a PUF-based privacy preserving authentication protocol, in: Cryptographic Hardware and Embedded Systems – CHES 2015, Springer Berlin Heidelberg, 2015, pp. 556–576.